

SoundHound CIE Interview Prep

SuperOwl Deep Dive
June 2026

SuperOwl — Voice AI Call System (Complete Deep Dive)

For: Full internal understanding of every component you built. **Goal:** Answer any question about how the system actually works — webhook payloads, storage layer, prompt building, escalation logic, WebSocket streaming, Slack integration, mobile app internals.

What is SuperOwl?

A multi-tenant voice AI platform where businesses automate customer phone calls. Each business gets its own AI configuration. Customers call → AI answers → resolves or escalates. Owners monitor via Slack and mobile app.

You built this solo during your internship at Beaut Group (Feb-Jun 2026).

High-Level Architecture

flowchart TB
 subgraph External ["External Services"]
 VAPI["VAPI
Telephony + Speech + AI"]
 GROQ["Groq
llama-3.3-70b
Summarization"]
 GEMINI["Gemini
gemini-2.5-flash-lite
KB → Prompt"]
 NANGO["Nango
Slack OAuth"]
 FCM["Firebase Cloud Messaging
Push Notifications"]
 FIREBASE["Firebase Auth
ID Token Validation"]
 end
 subgraph Backend ["FastAPI Backend"]
 MAIN["main.py
13 routers, CORS, ngrok detect"]
 CS["call_session.py
Inbound lifecycle"]
 CE["call_events.py
Transcript + End-of-call"]
 CO["call_orchestrator.py
Outbound calls"]
 OF["owner_flow.py
Owner escalation"]
 PB["prompt_builder.py
3-mode engine"]
 VS["vapi_service.py
VAPI HTTP client"]
 SS["slack_service.py
Slack Block Kit"]
 FS["fcm_service.py
Push notifications"]
 GS["groq_service.py
Summarization"]
 GMS["gemini_service.py
KB → prompt"]
 WP["webhook_parser.py
SIP header parsing"]
 ST["storage.py
Dual-mode facade"]
 end
 subgraph Mobile ["React Native Android"]
 APP["App.js
FCM + Live Call Overlay"]
 WS["WebSocket Hook
Live transcript"]
 SIM["SimInfo Module
Dual-SIM"]
 LCF["LiveCallService
Foreground Service"]
 end
 Phone["Customer Phone"] --> VAPI
 VAPI --> CS
 CS --> CE
 CE --> CO
 CO --> OF
 OF --> PB
 PB --> VS
 VS --> SS
 SS --> FS
 FS --> GS
 GS --> GMS
 GMS --> WP
 WP --> ST
 ST --> APP
 APP --> WS
 WS --> SIM
 SIM --> LCF
 LCF --> Phone
 Phone --> REST["REST API"]
 REST --> Backend
 Backend --> Storage["Storage"]
 Storage --> FIRESTORE["Firestore
6 collections"]
 FIRESTORE --> JSON["JSON Files
data/*.json"]
 JSON --> Backend
 Backend --> External
 External --> Backend
 Backend --> Mobile
 Mobile --> Backend
 Backend --> FCM
 FCM --> Backend
 Backend --> FIREBASE
 FIREBASE --> Backend

Tech Stack

Component	Technology	Why
Backend	FastAPI (Python 3.12)	Async webhooks, WebSocket, auto docs
Voice AI	VAPI (api.vapi.ai)	Handles telephony, STT, AI conversation, TTS
Database	Firebase Firestore (prod) / JSON files (dev)	Real-time sync, serverless
LLM	Groq (llama-3.3-70b)	Transcript summarization
LLM	Gemini (gemini-2.5-flash-lite)	KB → business system prompt
Push	Firebase Cloud Messaging	Android push notifications
Auth	Firebase Auth	ID token validation
Slack	Nango OAuth proxy	Multi-tenant Slack integration
WebSocket	FastAPI WebSocket	Live transcript streaming
Mobile	React Native 0.73 (Android)	Dual-SIM via Kotlin native modules
Deployment	Docker → Cloud Run (asia-south1)	Auto-scaling, 512Mi, 1-3 instances

Project Structure — Every File Explained

```
superowl_mobile/
├── backend/
│   ├── main.py                # Entry point
│   └── app/
│       ├── core/
│       │   ├── config.py      # Pydantic Settings from .env
│       │   ├── storage.py     # Dual-mode storage facade
│       │   ├── firestore_storage.py # Firestore CRUD (40+ functions)
│       │   ├── json_storage.py # JSON file CRUD with advisory locking
│       │   └── logging_config.py # Structured JSON logging + request ID
│       └── services/
│           ├── call_session.py # ★ Inbound call lifecycle (CRITICAL)
│           ├── call_events.py  # ★ Transcript + end-of-call processing
│           ├── call_orchestrator.py # Outbound call creation
│           ├── owner_flow.py   # ★ Owner escalation + SIP transfer
│           ├── prompt_builder.py # ★ 3-mode prompt engine
│           ├── vapi_service.py # VAPI HTTP client
│           └── fcm_service.py  # Firebase Cloud Messaging
```

groq_service.py	# Transcript summarization
gemini_service.py	# KB → business prompt generation
nango_service.py	# Slack OAuth via Nango
slack_service.py	# Slack Block Kit notifications
webhook_parser.py	# SIP header + tool call parsing
routers/	
vapi_webhook.py	# ★ Main webhook dispatcher
mobile.py	# Mobile app API (profile, calls, whisper)
ws_live_call.py	# ★ WebSocket live transcript
billing.py	# Credit system
businesses.py	# Business CRUD
call_logs.py	# Call history
trigger.py	# Outbound callback trigger
onboarding.py	# Slack OAuth flow
slack_events.py	# Slack Events API (whisper from thread)
slack_commands.py	# Slash commands (/superowl-*)
slack_actions.py	# Interactive buttons
playground.py	# Dev/seed endpoints
prompts.py	# Shared prompt template management
scripts/	# Migration, setup, inspection
data/	# JSON storage (dev mode)
Dockerfile	
mobile/	
App.js	# Root: FCM handler + live call overlay
src/	
screens/	# 24 screens
components/	# Shared UI
services/	# useLiveCallSocket, useSimInfo
state/	# Global app state (Context)
navigation/	# Stack + bottom tabs
functions/	# Firebase Cloud Functions (minimal)
root scripts	# VAPI debugging, log fetching

The Storage Layer (Dual-Mode)

flowchart LR
 subgraph Facade["storage.py"]
 F["40+ functions
All CRUD operations"]
 end
 subgraph Backends["Two Backends"]
 FS["firestore_storage.py
6 collections
Production"]
 JS["json_storage.py
data/*.json files
Development"]
 end
 F --> FS
 F --> JS
 style FS fill:#f9f9f9
 style JS fill:#f9f9f9

storage.py — The Facade

40+ functions that all call either `firestore_storage` or `json_storage` based on the `USE_FIREBASE` env var. This is the ONLY file that touches storage. All services go through this facade.

Key functions:

get_user_profile(uid)	# Get user profile with nested business
get_user_profile_by_phone(phone)	# Lookup by phone (for ANI resolution)
update_user_profile(uid, data)	# Update profile fields
create_call_log(data)	# Create a call record
update_call_log(call_id, data)	# Update call status, transcript, summary
get_call_logs(uid)	# Get all calls for a business
get_call_log_by_vapi_id(vapi_id)	# Lookup by VAPI call ID
save_session(session_id, data)	# Save call session data
get_session(session_id)	# Get session data

firestore_storage.py — Production

6 Firestore collections:

Collection	Key Fields	Purpose
user_profiles	uid, name, phone, fcm_token, mode, credits, features, business{}	Unified profile + nested business config
user_profiles/{uid}/sims	simIccId, carrier, phone_number	SIM card registry
call_logs	uuid, call_type, vapi_call_id, customer_phone, duration, outcome, transcript, summary, slack_ts	Call history
sessions	session_id, state, active_call_id	Active call sessions
prompt_templates	shared_system_prompt	Global prompt template
credit_transactions	uid, amount, plan, timestamp	Billing history

Key pattern: `user_profiles` has a nested `business` object. The business config (prompt, voice, features, Slack channel) is stored INSIDE the profile. This is a denormalized design — fast reads, no joins needed.

json_storage.py — Development

Uses `fcntl.flock` advisory file locking for atomic writes. Each collection maps to a JSON file in `data/`:

```
data/user_profiles.json
data/call_logs.json
data/sessions.json
data/prompts.json
data/kb/{uuid}.txt
data/phone_map.json
```

Known bug: `json_storage.py:176` calls `get_user_profile_by_phone()` which doesn't exist. Dev mode crashes on inbound calls.

Inbound Call Flow — Complete Step-by-Step

Design Reasoning

Why VAPI and not Twilio/AWS Connect?

```
Twilio:      More mature, but requires building ASR+NLU+TTS stack yourself
AWS Connect: Enterprise-focused, complex pricing, 12-month lock-in
VAPI:        Purpose-built for AI voice agents. Provides websocket for streaming
              transcript. 7.5s webhook timeout matches UX requirement.
              Free tier: 5 concurrent calls, enough for MVP.
```

Why 5-layer ANI resolution?

```
# Layer 1: SIP Diversion header (caller's original number)
# Layer 2: SIP From/To headers (PBX-level identifiers)
# Layer 3: In-memory phone→business map (hot cache)
# Layer 4: Firestore query by business phone
# Layer 5: Firestore query by owner's personal number
```

Each layer exists because SIP headers are unreliable:

- Some carriers pass Diversion header, some don't
- Some PBXs rewrite From/To headers
- In-memory cache is fast but can be stale
- Firestore is the source of truth but slow (100-300ms)

5 layers ensure resolution works regardless of the telco quirks. The tradeoff: up to 5 sequential lookups = ~1.5s worst case vs 200ms for Layer 1 success. But the 7.5s timeout accommodates this.

Why Firestore and not PostgreSQL? Serverless Firebase = zero ops. No connection pooling, no migrations, no VPC. Tradeoff: Firestore read latency is 100-300ms vs 1-5ms for local SQLite. But the system is multi-tenant (multiple businesses) and needs persistence across restarts — so file-based SQLite won't work. Firestore fits the serverless-internship infra constraint.

Why 7.5s timeout specifically? VAPI's fixed webhook timeout is 7.5s. This dictated the entire architecture:

- Critical path: ANI → config → response MUST fit in 7.5s
- Non-critical: Slack notification, FCM push → fire-and-forget AFTER response
- If 7.5s had been 3s: needed local SQLite cache + pre-warmed configs
- If 15s: could have done more inline processing before response

This is the CRITICAL path. Understand every step.

```
sequenceDiagram
    participant C as Customer
    participant V as VAPI
    participant WH as vapi_webhook.py
    participant CS as call_session.py
    participant PB as prompt_builder.py
    participant ST as storage.py
    participant SS as slack_service.py
    participant FS as fcm_service.py
    participant CE as call_events.py
    participant GS as groq_service.py
    participant WS as ws_live_call.py
    participant M as Mobile App

    C->>V: Dial business number
    V->>WH: POST /vapi-webhook (assistant-request)
    WH->>CS: handle_assistant_request(payload)
    CS->>ST: Step 1: Parse ANI (5-layer resolution)
    CS->>ST: Step 2: Load business config by phone
    ST->>CS: Business profile + settings
    CS->>PB: Step 3: Build system prompt
    PB->>PB: Choose mode (personal/business/hybrid)
    PB->>CS: Render template with variables
    CS->>CS: System prompt + welcome message
    CS->>CS: Step 4: Configure features (recording, transfer, whisper)
    CS->>CS: Step 5: Build VAPI assistant overrides
    CS->>WH: Return assistantId + overrides
    WH->>V: HTTP 200 (within 7.5s)
    V->>C: "Welcome to ABC Clinic..."
    V-->>SS: par Fire-and-forget (async)
    CS->>SS: Slack: "Live call from +91..."
    CS->>FS: FCM: "Incoming call"
    CS->>ST: Create call_log + session end loop
    Every utterance V->>WH: transcript event
    WH->>CE: handle_transcript(payload)
    CE->>WS: Broadcast via WebSocket
    CE->>SS: Stream to Slack thread (throttled)
    CE->>FS: FCM update (1.5s throttle)
    end
    V->>WH: end-of-call-report
    WH->>CE: handle_end_of_call_report(payload)
    CE->>GS: Summarize transcript
    GS->>CE: Summary text
    CE->>CE: Classify outcome
    CE->>ST: Save call_log (summary, outcome, credits)
    CE->>FS: FCM: "Call ended" + summary
    CE->>SS: Slack: final summary + action buttons
```

Step 1: VAPI Webhook Arrives (`vapi_webhook.py`)

```
# vapi_webhook.py – The dispatcher
@router.post("/vapi-webhook")
async def vapi_webhook(request: Request):
    payload = await request.json()
    event_type = payload.get("message", {}).get("type")
```

```

if event_type == "assistant-request":
    await handle_assistant_request(payload)

elif event_type == "transcript":
    await handle_transcript(payload)

elif event_type == "end-of-call-report":
    await handle_end_of_call_report(payload)

elif event_type == "hang":
    await handle_hang(payload)

elif event_type == "tool-call":
    tool_name = payload["message"]["toolCall"]["name"]
    if tool_name == "notify_owner":
        await handle_notify_owner(payload)
    elif tool_name == "owner_decision":
        await handle_owner_decision(payload)

return {"ok": True}

```

Step 2: ANI Resolution (`call_session.py`)

```

# call_session.py - 5-layer ANI resolution
async def handle_assistant_request(payload):
    call = payload["call"]
    caller_phone = None

    # Layer 1: Diversion SIP Header
    # Some carriers add "Diversion" header with original caller
    # Format: sip:9999999999@sip.vapi.ai
    diversion = call.get("sipHeaders", {}).get("Diversion")
    if diversion:
        caller_phone = parse_sip_uri(diversion) # Extract "9999999999"

    # Layer 2: SIP From/To Headers
    if not caller_phone:
        caller_phone = call.get("from") # VAPI's normalized caller number

    # Layer 3: Phone Map (in-memory cache)
    # Pre-populated when businesses register their phone numbers
    # Key: phone_number → Value: business_uuid
    if not caller_phone:
        business_uuid = phone_map.get(call.get("to"))

    # Layer 4: DB Lookup by Business Phone
    if not business_uuid:
        business_uuid = await storage.get_business_by_phone(call.get("to"))

    # Layer 5: DB Lookup by Owner Phone
    if not business_uuid:
        business_uuid = await storage.get_business_by_owner_phone(caller_phone)

    # Now load full business config
    profile = await storage.get_user_profile(business_uuid)
    business = profile["business"]

    # ... continue to build assistant config

```

Step 3: Build Assistant Config (`prompt_builder.py`)

Design Reasoning

Why 3 prompt modes (personal/business/hybrid)?

```

Personal: For personal numbers – "Hey, you've reached John"
Business: For business numbers – "Thank you for calling City Dental Clinic"
Hybrid:   Auto-detects intent – "Is this about the business or personal?"

```

This maps to how people actually use dual-SIM phones. A single business may have:

- A published business number → always business mode
- The owner's mobile → hybrid (could be business or personal call)
- Emergency calls → bypass AI, ring directly

Why prompt templates with variables and not hardcoded text? Variables (business name, hours, services, greeting) mean one prompt template works for ALL tenants. Changing a business's hours = update

Firestore, not redeploy. This is the same pattern as Amelia's Entity system — define the data separately from the conversation logic.

```
# prompt_builder.py — 3-mode prompt engine
def build_system_prompt(business, mode):
    """
    Three modes:
    - PERSONAL: Owner's personal assistant
    - BUSINESS: Business representative
    - HYBRID: Detect intent first, then route
    """

    if mode == "personal":
        return PERSONAL_PROMPT.format(
            owner_name=business["owner_name"],
            greeting=business.get("greeting_message", "Hi, I'm your assistant."),
            spam_rules=business.get("spam_filter_rules", "Block telemarketers.")
        )

    elif mode == "business":
        return BUSINESS_PROMPT.format(
            business_name=business["display_name"],
            hours=business.get("business_hours", "Not specified"),
            services=business.get("services", "Not specified"),
            faq=business.get("faq", ""),
            booking_link=business.get("booking_link", "")
        )

    elif mode == "hybrid":
        return HYBRID_PROMPT.format(
            business_name=business["display_name"],
            owner_name=business["owner_name"],
            # Includes intent detection instructions:
            # "If caller asks about the business, use business flow."
            # "If caller asks for the owner personally, use personal flow."
            # "If unclear, ask: 'Is this about {business_name} or {owner_name}?'")
        )
```

The prompts include `{{variable}}` **syntax** — template variables that get replaced with actual business data. The Gemini service generates the business system prompt from the knowledge base text.

Step 4: Configure Features

```
# Features are per-business toggles
features = business.get("features", {})

assistant_overrides = {
    "model": {
        "provider": "openai",
        "model": "gpt-4o-mini",
        "systemPrompt": system_prompt,
    },
    "voice": {
        "provider": "l1llabs", # or other provider
        "voiceId": business.get("voice_id", "default"),
    },
    "transcriber": {
        "provider": "deepgram",
        "language": business.get("language", "en"),
    },
    "recordingEnabled": features.get("callRecording", False),
    "endCallEnabled": features.get("endCall", True),
    "transferCallEnabled": features.get("callTransfer", True),
}

# Add tools based on features
tools = []
if features.get("callTransfer"):
    tools.append({
        "type": "function",
        "function": {
            "name": "notify_owner",
            "description": "Transfer call to owner or request human help",
            "parameters": {
                "type": "object",
                "properties": {
                    "reason": {"type": "string"}
                }
            }
        }
    })
```

```
}  
})
```

Step 5: Return to VAPI (within 7.5 seconds)

```
# Return assistantId to VAPI  
return {  
    "assistantId": business.get("vapi_assistant_id"),  
    "assistantOverrides": assistant_overrides  
}
```

Critical: This must return within 7.5 seconds. All non-critical work (Slack notification, FCM push, call log creation) happens AFTER this return, as fire-and-forget background tasks.

Step 6: Fire-and-Forget Notifications

```
# After responding to VAPI, do non-critical work asynchronously  
async def post_call_setup(business, call_id, caller_phone):  
    # These run in parallel, but don't block the webhook response  
    asyncio.create_task(  
        slack_service.send_live_call_notification(  
            business=business,  
            call_id=call_id,  
            caller_phone=caller_phone  
        )  
    )  
  
    asyncio.create_task(  
        fcm_service.send_to_user(  
            uid=business["owner_uid"],  
            data={  
                "type": "incoming_call",  
                "call_id": call_id,  
                "caller": caller_phone  
            }  
        )  
    )  
  
    # Create call log in storage  
    await storage.create_call_log({  
        "uuid": business["uuid"],  
        "call_type": "inbound",  
        "vapi_call_id": call_id,  
        "customer_phone": caller_phone,  
        "status": "in_progress",  
        "created_at": datetime.now().isoformat()  
    })
```

Transcript Streaming Flow

Design Reasoning

Why WebSocket + Slack + FCM in parallel?

Three consumers for the SAME transcript:

1. WebSocket: Mobile app live view (user-facing)
2. Slack: Business owner monitoring (admin-facing)
3. FCM: Push notification (interrupt on phone)

Each has different latency requirements:

- WebSocket: <1s (real-time feel)
- Slack: <3s (thread update, user tolerates delay)
- FCM: <5s (push notification latency)

Running them in parallel via `asyncio.gather` means the SLOWEST consumer determines throughput, not the sum. If FCM takes 2s, WebSocket isn't delayed.

```
<div class="mermaid-chart"><img  
src="  
/a8bUC6CLUuAtaddTFgDQxYfWpMCClFKRCqRQYkAIDVq+sQMEaaLEGWjA0YBem2CKpViCMLTbUGjrFuhiwMhSszBS2TkqxTpliHShYBwJWIGDly  
vpfUrftyvY/7I937/f8/bva25KWvWs53NprWd922w28MQTTzzxxBNPPPHHEE0888cQTPwL+vc8m4IknnnjiiSeee0KJJ5544okn7sUziX3iiSd+Q  
CSB3VUoDJXsBAAt68LBEoa/mfT9cQTW5yK6Kci9qyurKKY6gJAGjp9jcnRph99NL1P3I/XilPk9hUKRSU7/TDSnvi5IQ1tQ1UkQeo/Pe4Tn4rf1  
CQ2dAyNI759wwiG4zi0oUiCoDhFt7x4f0/q6ST09cNPJfSjEFkCTqjuFW/3vbqfhrhMPi3bxijGJa/pyhy+7pEYpRkWMGbYrzEtwyJwr59Izita  
wd7Tx06XY0jcEYx7G0nY1PAUJQz45PnY8/UFQr79g2nMLGh0LWfyUka2IbKEd++4ZxqHDX93rhnlIIug1PfL0dLI7evSRwnSJIKSQLHMIygdL0EI  
HK7ukSi3lCC4rZgKBInde970QYAABiLGBqDbf8jBN2Qi09KwI+PxFEJQRLiK7d8J6LlFFNLSG/fCEHrHtkIiOP937FndnWBxEip68bbRwS0EYRJE  
BiK4jT7y2VkpYl6qR4H51Rdpbb/oKSk6wL+CWTcja/jv7ZvCx1D4jhBkCSBoyhG6Prp7Uvvi3NxeEoL/msffEd3ZeH4ivYzMB0sAyVAb/9g0j9  
56Yo0hc2JIXu11layI5hqI4/Zo+PYagr3RTzXa6XPrpE3T34LY9+VFwh937fp0Nna4mEN++fcMU5+JlqasS376hpKT33Q/lzZvfWKxHIoLQnfnu/  
9Vs2KiVoM2ZuvsL/lQ5hvj5Sf5+L5YTie7rm1Wk5Yo73uaiw/o/no+mS7yLsxaVUDYwem19bRJ1yfXSLz/xZN6GUriAHXy62os082X/b+LQaPZk
```


csXdGw2m1mrBiV5k310qERh05tJwC55BGEHby/mKxmK5cdvXmXxLc40vD+bVbHsZiMsltcPXSocltlvvdym/1Yiil085NG2WoHLF2sxxUm9N3p
e0fTBsVZD/oywGLVFsvt5750BwP6JfFMZHrL54otqbXaP5+UreeNipQaUyP7MFqXKebxz/MwvR2uBcDtKy3DuZu2uBbs3e18p1wIqKf6nHWYxkp1
Sfb/+adGKL3co312+Hz/VerCq+njUpFHu3ftrJw8mh969JH4E5CHmsjJgFSNYtfIQZ+X74sJYzMaQIwg+P6Jk25EYi2nK/GC+16JrWx2/GfM0j
fDDHyh0vs0efGGcxNW37d5HdPbY0J/SNhJLCAKXIffJyyJdLCFQaHx0X/oyS2M1ms1m05Qqc/3g/VrNRb/QlQ6UsJazyH5BrPYDFgK8Wi0yiRyNQ6
xwzbz2uH6eYb8TLZZq8Hlps7zDUi0G9M1tN6uWc4004id1sVMegdpW5J4qiZ136FCL4vtgNZYr8jjHMR00q0hJHJ13eTke7GznRRK7mU8m3z8i/
lJJ70cP6K3zCfILpe4i1l2NXLJD4ccdJXkKfKxqVnPxPmVGV09Muv4AHZxJPYL+K9C7IhpVvK5dwGiy3oZ0Vy59BH7QJPaHxhe2mXlJ7HIyfl1n/
vMejZwEXKvp+N3mWf+ai10XPvGbuBJfydcMc7rkcz62uEqZ3qyKzDyg258gW520W0I7KsKM8Tdez+HY9HxZ4lhdLnmXl/Veb9Xz2uGm1L+6N
earJboz36xehvUasRw+i0mwTpcQfmuK7VNBnW5WeRptj6crzfr2UcULZCq2JDZarmElCp852WnwItJS2Z2UZZFng7tpuLX047MsqIssjTfHG8dc
14Su9nM0ZAWWmu2Wc9GLb6C7CPx1bjBsqIsyzxdEw+foJbTjszycL0W2VqLVBFIh1m0emIvARSH056dbZabUw3m1LPZHlRlkw2xjZGi81mMx816
DJS4et1vlpGkFKFbQ7Hw5bMvssIUq7VL6ZSVtM0W0FKNVHm2VqLXK6KremyilbVy0CuIYg4mA6bflUi/6k3rNMlKNd4UW5NVstpr14rHczMajbMR
kou2VpNHC50ur+eDcQqglR5mwfparlczfM3ExblMtnb+Zbk+Vms1m99M0qgtTkpkXywhSqsR+WazWYxbfAWQCiukedOQy6FYgsRd5XLUgtZx
cMlM3PF7wyLHn2qS1seKx5NF14G9c4s5XP9e08id1M5BJUt18WC5LYh5i2ng0bda1h05NRS6xV+P/x34yabBLKVYU6PwTBQ36/mouWMBXWM7s
81mPu6I1W0M0h/KtRJSYwWRpauVcoWu03fzrZesK9K5bQnVxGo0KIo904zv2mJajcjpZMKdr2YzQtuzhvKzWLczISRp+lmzpf4n04Xq0crkthzW
7F6GchVBMpsYzDNxHvY5CtVeFCyyjMsy0mLUcJrTfEWQWEIOVafbtISD4M60JqH4sKZnt8BWAactsbLw8yVaq4uDkc/iDqnp1Jf7RP/ufj4lcv
QwabPmw5CDHSHxL3YMC/7DUXWsxYyFfLpWgVf+SMe+wu8Fej+USLPnB7EYclSc26/mwzr08L1sszXZ0iVh0B3IngQrfzKRhs14uluVNZrNZvgxbf
LUMD6blmwJheJlOIHPyKCuVkuZbMzK5S2mu0vDM0ySxy3GTLK0ZFkwerLXKlbr89Hdb0ESE3ujBebzWY+7si1C74Io2vX6znyo+jd9
V+b1bQn71heq9XHyP3/ZDRvFuFjZdv0AhSa+RNk1y5DjJX0LUMTUXenW9ZJXJ0nsej7gKxU+szFFyCPsIk47Ny//+X/73/EvGARFHM5Wm54
EqTPpoHR2Yk6110FHfZ/jgKhngxabJlgKrYGs3W29iyUqsPj+cr3hJZfXWbmZPE7sZwyCNQkufnnDxDrmNdvfS248PS4uAsVlq9DJp8FYFjd9Td
D6gq50+TEi5yrdG40FTpCsLBCmz1z9L5xNwmW6chVX/8X/93xcYlFcmPSLMZ3YocLPwS6fCaBE13vT5dbGvYt8580ePBiwncakf/jPj+Pg87QlX
/6fGm+HxR045CyZzASb09Vms305Mc5nQ7UeyWxv0uARq0zadpMpmWmwzwoecM6JHuip1V3B3EvvSr07Ci+WoM1xk377K2YUfyfQjRf1lkgU
tp//8pmg+abvRld7VqrbJPY9aReqcjJlWbrD7K7550aUmtNV+vnz3vsch2oej6pVU022dU540GuP1ZrMcieut/9bTzrW0pSk/iddfM6uVdhL4c8
gdp2PUqy3K24r4ay+XKNohajcR5AUvpZiWjCN0cvcznw0Zrutm89Hpb5gzYXZurIY+U5dFitV6v5k0xjFTro/lytV4vhnL5Mo/Kcidk0+G2nnXo0
u5rWC5t00YFqdaH09l83Gy015uXvU41zqaK1u0xPK05vXNo1fPip3j7h/NJ6/nA35vpteTTi+zbMuCnozmaaOCsL35arPzrF6aNaRcn245hBTpu
6wn9TKU9/OJiWfftB4l5k54neB5ADAvXWv3BmWu+/zZr0Z77qdZbFnaw9Pk9jFgEX2X3WLv8Q+XrL5p4U2uX5nKp92moPlZnFqjboXWE3Ltk/6
+NqtFunexSDZpqSvWxM51iv75C1PD0au0T0p78XTGtSbDi0D1uLMkyR23qn3Ckx0Dj3rsVedXw+6IzPjWJ+93NV4PEKntdWLEd5igXw5fCbAsu
jkbTB3G4W82m+w4K2rDoe0fT9LJKceY9G9jYmWwB2IXePq+qRlTiVupM7C6zEcebzSMA/IHX7sd3p+alH8r1Rsw07hsx7rLiDPQtazDl0CgFKVb
3RGL0UpRI7YLId8aaF0L73ehblaDY/s/GoklneSsZ7UxChI9uCe4lREj9T8QanLR4Gt2J01HsulKpw5sPu+xK6G/H4IVtNmDanUJ1tHfey0Zh3+Z
0X31Wfz6TnzocX+azPvsaxQfv7zpVUfLM7d900u8x4VPsVyLFcQAKRCy83B5CQvuHLpG8A7tXwx0xGnPGufz6tDErsY1uXBjCQmh7D80C3PvBxt6
pn3+K0AaSd7uVfC0fVHKBril2YV0Ux9LQaifKmsB4msvbrNPEliV6Pm/UPratqsIgBiUsY2e+OiaeY8Dsw7dHnbq/WkM8iJBE6+x0aH5fmoJYsBk
8VqvV69dNgSunt0f8v1eJxR8aUba5HzCMHNN87Dq1Kjeoc9Yw//LKjcoG55EsHyNeVnY0BcceD9i00nUW9VRK509pFLwCsU09pFL5mLEPdb7
86M8zhWI5ntLVZj+Xi+eTkU+cFiPW0WJbE5HuG21BXg7sJ0SRInUrILAIArhkoA+LYtkVJWHgKnKCL2vAhS17RjrlXJ05tNPduJKUXAAABHsJcZ
9/umUdxAKMBUJJjYpDSOE4DUM62Q0QLQ5S2W7gdJlCIXEtFzjJQ0EAZRgy9d1rhRUwDAVI0457tr3K3gtJkgAABK6XEAyFagBgFE0kob/foYySgsKQp
Gp20QBGlzkuAADH8d3D28YIDEUxUpIoSDGCxDEUJSSBgiTKrQ2AAQAAACil6Ry4lpsU0QYAAQAgSR5FSvyuhxT207LKLCPq0Le1AKAULcFA0ez0Pg
hIWZG4IAKhg6Fk5EAzHIYk0m7DRHMKAGMKBUKs+PYmclTQl0rfYJdYdWAETe1pUn5tkRxmXorfjXBFL8q/sk03BYDUM31KE/1bLLELXwUKAAB1F
AkPhau0BEIAz2kbfvUduad00ENMWzLJoEj06GtXiqokrmknL070YyWq420mgALZ4CZhnZtg/5rNvefaFNhU1T5CcEw691YIbY0QVBPEARBTQ37
efQkyZJkgRBDABAaoZ0VnGkoPuFkVaq08m0FLmkSFLhmAAAAke0gioIX3bwlCydQAMAFgYI4vtDkG30EKJAcULKZdKv0kLh0KuWryG0q+nLw4gGp
0w1BTx2BR8AQtvGFVRpdqWZJhrytKekyhleMGk12DA/Y5N/+vv/EUFKZphj9XPEJk2SNA7CBLbief4I1JikcuLAkAED12t7FBLYTakDq0YkgEPcN7
h14hdR0dmHw7MLCMRkLk/jRQkPb5X4XWNix/ZTAJRTJy0rYzclsvPVIhcsVpnXc+k58aGFCCTJ1Rt9zpGM3KLJj3m0m+p8ClwyQ78UUVci+eP/
uA//fsEo7vx7UvHeNip35Fm0Le51XiGUaomhp53bhfEnYlTy2sL45iCeCaZ3rgeMSRy5416+8K05a+0f1HWJ3KLJj3m0m+p8ClwyQ78UUVci+eP/
hMA4Ce/K2V1nYS2l+x9FNf3g3FHIRP3j/7J7/19gLLzSt/lcSCJk93404KhUzDPHSM/LN92LQt/GUgAcVxAKdRjBIEEq53LIEa3HRj24VDFWfVg
E+RqyNX2j9XCtCF5yqh+YTAQCKvh1xU17HHgvYPi9WRDEcRwGAEAQ54ui+SAsTNKXyN465je4tC9UU4orq5H1Ex6tUCL91742cppHCH1CkJSmar
qsMnkZR9FNkaYqHAKCaoCS0AKRR10IUCbdfT6IohaivKjYAQBrj50WQobvW4ihKMALf34GSJAEQhDGKUVRTACAJCGJqyY6SVIAFL2gMPHNdt9LU
AqNp735ZkZPliqCQCK5badhk6/aweAYWGUAD3Cun0zQAAjCBw8MIIGMqL7e72oiAC/Cgjw8kbUwwoQRLghDEAmQRWt+9GgGEQhgD5D97TGWaAR
lNps23ZapFLlktqer6Y5DHZqVxy6RA0fpfnlukkGuCZowAcqozQcQLPFYtsvWTKglV4795mTgfo7zxCEbpoARmmur+6dwm18yrCRREEIUHQWAAit4
QKCxCg0ohQYUc2H15J/CQeKIj+ltwNfX0q7fQYAor7m5N+SN5SYpKtd6fdJwpQ0TdcU6pSggu7fVIG7E0XBclTQFIKZLa/d5yLHwVQHk775FGiu8
N/o41ktB8k8QlI5XbdfRFAADUDq3ygh+5DUfakVuC112W2MhUcW63gp6jie4uKAQQRZb1t0zKQKlNHzacEOJuC3088u2vof6Krfc43zt6dIzaEp
Et99XcJj1c0TdekioHFBIUDzfZS1XfCtGsQfdsJdQ12E0EnHh3ca519vdtUGoYrffvtwUw9DYT+ANZYgJgkDTKEoACJRRJNxy7KBLUyU4px/
7NvoCnNwgiI4vAU0dYVnEdR8p5LjLNK1LW450dqt/9I1b0q3YvY4d0GoVuy2tZj1tXunVGvQVqx+r19KyJdJEE3RWNACQyAJIhmXaoGaVe
ZRqXLwsN4qL3dvtn3LnmMRAQrQZ71t+n+Ni18Uv+4ZC/iCR1Z02EwAQXnBcFQ0A2NL6RxcwUjIsyTAj1zR04880jeNc9ZSzeRxnJ2x/4AhbEHRd
F3lrmUjRWH5ewf2YfQ98eILafnpXixYje0JcnUk9e9r/+ihXdcFYoiQYIrtRjQ0+3Ys608asF3DB8eKAHA0bNeBcpps/Z0+aUeqFvZdSi+17rf18
QiPSd0R7k5iUabr4PjW2Zf42zPCUwUACE1y1aPi0QLotnrTRu2LZz8VJz957MUE9p2/y4JiMIwAZKhzhgUW6rQx53A4LAiU76z7QYlclJleJGmE
hC6XK8Qp06+G1J9fA0wgARyn8xvcmUGAMLaLftfwrDMfvBRQAULXJLQb2goGSXMY3Ucb0g0palxbs0yPIANG7+ftLcvMC/GzLmZ4Y28mm7afu
onSPRk2LBI7PWH53jYgpp5/61he1zn+MFDiuu6ZHD6CdZ243uqWB0DR9XJdgW4MmgtVdC0139dwl+fRggh1EqmblN9lbnJ+2+8y5JjX3/80qc
IRcW8FoKmx2La+NBS6paejVm+96yc0+HpAe6SguKYKu224sgHs7h31PqfsSViLLYh2rnySk2sUAGFRUqm1Y3T6KS1buBziCU00HIkLzgwTIY5HNF
xtCsUNGS2377CqM3fX986ADExQ0dNsNexdVTA13GNsJmJpJmdGJ7/YHBvYb3kboLWxH2JcWSPN+gUPB0132nU2hQTLIEI07H9Nng+eTOH/XB6ruKdX
0cLMFIyHDymONcLQWlUvPQap3ZLzB609j/5XmK1Bc3Q2fVc5E9xSRiVh06LPMppCsF3pq9JVsTpBUKc693uav+0ke0nBVJRQXbX81i8nDjukMzf
9M1q1wQVnyZLoJvndNKK6GdQMYsLEOBqXsi5jmNXtd+13Ahv55X990+3Xe+KACUQ3C3hDkVg9gbvPZ3koax0TAVAGtCFsKDSz8id5rzz7LghBz
8aKx6A0je7/E9N00DCWrrvEfI/wsNRtcf9y4jgGjJK0vutbQuLYPooTBETH2Sd3nMAHjgpXyl4AIwg0vWlAdhBYGL4fvN2buG+r96hbQdAQ9K5N
IV+kJLcew0LqNdtrCGpKYqitCPfCY28j9xx6Ec4Rb3zKZVPFCUoQebFtN0Ng1QgCvIw2xUhDiMgKBK1iIggJ5h2PmMMLTcJ+aaGLBUTf8hn5oX4F
byAVDU0/rK9EE9miEnB18Qjvwpykhc+de05b2n7X0vphUEAMU213I0ojIEkCPS95JDTLfrqWc65FqeurruFTUw23g10fimgJ44FTGcU3fS8Ph069
ulDn1/qwrsGy+G1YfAqscant1/s+p27t700G5QzX+3h+81ZYALLuARJ4L25o0R10Q/+2ZV/SJWAmUUAf/1n/Y02S10MBGKysBf/pef0tHCYnJST
oQnW3EMRCr+/qeWKTrrHCUpwgmH7rk4Eds4RdpigCKnb1xLM4jBKkCjQmXoI2wl+ahBVyQ3SL2usqShHwBFv1bqsq8KUbUPosixhKrgjm3ZaAPcX
u21e/at9KAKSUDOP5D6vpvr3ME3NPuIySh0YIDh2I1LB7zKqd0Qs9w2r/AK4fS2pXBvNjWrx5zmKfY/XEaymgQdL93UwEJS+c43iHooD4XiIA
QCAKfQmcS3L9vCCRE65z/4GRVY74NRu5jnsa+YxP3CEBww7e2M+B5I4TjFSULu07zRQ1/GvWqP8sPxNuCOgP93IRYSRPUGujjzQfoZd0JU9zakSH
t12ZSecDv+ereA7fNjXRNQFE0o/cpUu6h2Pf3M9wipSD0R7K5ivbaxCP0WRVGUwHGGFIGIRPZ+K1KAagDGKlQ9btecvqjAADGABaB68UAKmRb
rEiYgSFpht00rPH7fNjXRNQFE0o/cpUu6h2Pf3M9wipSD0R7K5ivbaxCP0WRVGUwHGGFIGIRPZ+K1KAagDGKlQ9btecvqjAADGABaB68UAKmRb
n6x4wki2/IwSR0Witr0gWJo/kZbQtG41Gkbbrrzn8VwmzimC4IqZTYv8v0EAOLw9ip0FAVirm92yJb0J6Rau0E0n5mX4lCAXNKK8q9TSjtazhXYP
nE6MY5YeLf669dy3y2LfhZpKlPCEuUs2sUETCG9rr5XiRtTba1n0RGLqtTDcoiaaP5uMKpTdanAULsndiR27SAtadv1BF57NU+XTE7vt9naXH4q16
PnCsdu36kCN3HFVucSjQb/YUfcpJy/27DARjP7Ac3rY9BncEK5MbK4kpztayVFQL12H6TbC9rft+q+q5UokLrtZUYRKgihqPtgFJdUroWyuylGy
Mixvh0jswN5qKSeX6+5YhPZhrUyFAMxfDzhA02WSz8ZHzF5JsqL/yEma32+2GAD+EN0pdrKkgGianYRCKAGK5YXqsMysS0SUm9yIZT0PLCghV3
dt05LwoyDQ8Kn/V/6zdn9WY+atXZCKCPQRmZ6GFncHJZQh/TZ5CtpoemC0W22pK1qyFHPQYpESXR/01pv1rMeJ9Fv0TRCjkydFVAAunJm
JXFXM96fAVByLVW7vTqVaTctyZ/NemIFSjR9d5ksZqNWqf/VKFE1y+q3E/kEL1L6426LLI0u68Tn0Pbv/GHdb0EvBe12J+CP0ux5VK5yoq9fz0bd
eRaCsp8szddbHnMWsvlcrVGs3xj0D/v/kuzglTYeqMuizx9KDC+mrb0M1Iq19h6ryOWS1Wx97KcjbXJd8bz9fJlKp3Tmy43m+W4US2VKjTFvHIW8
7zd168d55HHZGWe+BU9P23yhnxq181FTpstlWj46U2D1MmjKdBAqfKFNziScjVoyXQKkyjdbvZPTTetzUav0lghKBd2r4X+M+5nm/WK0Fenevn7LY
ihwS5UqLbZg/9vJWGRV1vlqeS/QL+vlndNgSqwHsfVvDl+Vm3qGRM1036nWZ2Lxp3q5rCvWpVW0RZdK5RqbeZ7cjpoiJwvyrJcL2WRZVn+3posN
pVfPNPqkgwglr7A03XK1X2/Cz1hHPWoxZLZ4cLsPpW0j7x867n68C045cZ2ldWB7J8/VJ6i9tXUFWGjXxtGj15c322SH1JCL5Nl5vPcdXaal1m
nwY0GmU08dZhy5X61n1vQLJ2REyqVfKhTtrygKpGsws rcSDyL34x7jTYMPTZRM88X+dZiTOpe+TVv3i1lG0V+LQ26Woky6cDs54NGyK7NQk8y4rN3
KMK8qz6Ytxg6ew0CLb4BJvVWVK7tK5700zR7KOR7W4D3d84KRHR00Brolqm7ghx12cyHcrWE1Cq02001aKTMnkZ/NR22xL3Tmo97u6GfLDab1bheK

dNZBcVvKEdKnBGROSH+nK+LHLuii2qRFz2bS8+ZD73qvzbLaUemt26fFZvjxTkz73eds+VdKnyikctJR+Zp0jtKhWf57DyNW5d0RuLhp3YpZqfi1
N9mHq9WL4N6DYGqPJxt+whQ4VvFhzfLEnYZp60uzcuW9LFCpbJf52YkP4q7bD8rz13e1ZEtG0Ld00diqVp47PybIquvazPn4yw8qbD1ZucsFnFy99
OpHgsnKnbyD3HPj3vLQpnmcch9cHLCYdDQ0aAhW2vguLqGb0EHUMpsvlyzDvn4t46ghfeQRcphvBU7F0wqrs1L0j urptTKwRfMcQJL++q/xYEXtMw
lcyT8QsC0u03UUV//VuR05sP/Ls84+80pfTjt8BU0Y/yiWs/Hreyf0X9bJxPVGw+7jT4SpmWw1lmtX5pyVtPuRh3mmIVaAqQnnvnopfjERY3p
K4Y958T+yoU16z/MbCaNsXm4W5kS5viu3Cg0/iZM5NLRyWrfE8/Dq1+Bz2PavEPnno/xxI+BG5KzGmINwnmeT1TVp5X4zcdqyCPFYcBmPW2ThYbnx
rMfic9znU98J9wY4l1LHD4W7hLzPFFW3Ai6Vq06PCoa508M2H7zY8W7Czu9D+93/s7I7IMB+/299/GcZLCUe/h7fyP4M1LMZ74meApKQvDmBNW
y6HSUfFvB4wLtFD1XvKkzxxjFDR8vdb5bnYsKs+koRtRwVv6vKAAAGALEQVRzn21UngxbEcSWInSD55ahl0Xa6hewuhyH8kioc4CaANovP57Cke
wupqzE5K8UBJTTbvVnh95NQKESJ66JK/+oy2k8UwN9o2f/gJPAHB5qd1JXJZu27Vhov/ZgwCee017g0n64XuVe/y0SBZTzTQLtdse1767quITn
46PlrcvIM+BZaaKRgXtPqj2F01wnngIuGoH6mcT8WZEthkKGhf3u6HU/6KJzx05QAUZ+B6Fyj8Wnmnhmoq7hk1ql8f5PPE98IFfeVcVvTpdghLdG
P2oH7PX82GDZ3LRlMVR5Hm5M15c3aD0FiwnHb6CQIVt5G5e+EcSxnpYDEYXW+NfytL398Zn8e0dbYVqVSTm9c25D3xRVEK0atxvVqp1uj6FXP5i
ar6tBI/A8xHDbYCSFVsdK52hc46dKVSrfGdafHCyKJnPx6f5zqf+E4oGuLFGk9UqjW2eWUP3T0yeuKtKAY6ps1qpVqL5SulCz4xYpTzXIrFnpvNZ
+fRTzzxxBNPPPHEE0888cQTTzxxF++4YueJj5544oknnnjiSeeeOKJjZ4bXyijTXXLFwiUNPxrD8We1ZVIFFPd70dZASK3fwfBTzxxLsJdW0j1
w7EXl9jCJTpv/Hw9TR0uhKJ4pp3z92xo3Gc0jWd4PWH2D7xs4WvEyj0SKpu9N375Db2zK6uKRxxUIEKcExDyjjDe8rgE++GLRxPfhPSE0nr3F5/
jXxLUMiUVRxPom090Ua9CVG0PpH5HfgaeyfAbebhWyYtJriLUEALk4LjI7RAkAKnyzM76yN2A5YJFqK6DE/2Lllyv17VK367GMPF4vftNgSwjCD
t71fJxCGt8H83GrcXbaLV2rL0kf+NCi74LVy7Al1koAFbbR0TQ0bD7uNNHkMzbfcYfKejHpnUSW5UVRFEwependyXeLyfagrMN5qnStWq42p1cbP
Bl0ulat0mLr+LDG9XzcqfMsl4pydjKh3BrNj/egLScdmWdZXRh5lq7V2KH0pMeq9U4V7SLMvzPFstAZRrLM/zPM/TlWojl6XKJl98ZsGDMHxocu2d
aXQDsw7tXcPkr8eiUg5ndnmLqW721a7se0Uc6602ZU5K9jG9bTZrXMP0omodQz5od0DjrvZHK55zU8gaE72AXXpovRBY9s/SfTE5VYF5p4aw1
2T/3e3VK1BIw3I6aMq1zDrtPV61XJBHV9s7oZymq9Ua3xj0fgbHmpzERR+H10VcPzSWL80mfCKhrNgaLzevFdH3pGE9G7bqdBmgVD0+p7184/DHC
8rpWValTg+9gyn7rvYv07qJeQDQPEJW/FUuUu33/55M90WT4Z71vYadqoLBC4PB1L2NZYrJQQQeXzdT2c5YU7gvJq0RLm3zUZeJSWLQaP2Zku5G
fbr8CfJ7DaQp01MZ5365weqq9n6k6GjtxWtY6RSdYz2Pl6EKnp3ctRrL9GV4v6NBtP8klXlM02m9VLhy6A+ZOD6teLoSzupGzaKMNJPL4c1G8Hh
SeDvhjJfWQXaymrVqpT8G9Se16MaX57XmrpbKrFU84AP99dV0S8ykUd/xYtGjkcPZdathPKXvmcNuLuMGf3ZQ+5vxhZLYaa8NOQsp5PvVELh3
qmVxPF6s5L36NK0ivV8Mv3cGjfczocw3xrcS+yM6l00+xA6Wm81qyJdu2unPtzn3dfAu0t8viV002CtHtb6jvZqPB5N5PsVXLt1Bw6xVg5J81Kf1u
F6/HRgfU76cNqpI6QMPDN/i++44U7iomK8lC5XxuXz0eBKuaV3whvsQBFb1vMBXy7TRRPiL5PG3h8+IKKvVZNBnCWGLHJ6/uy0Wb8t8GeUryaNa
k68foFT9/1zSGIXAxapvS+++fQk9g2u/9MN2qvw3suJcUFh4Zem6f+rFSXTZZiJlJPL6ZjGh69g69fpjJyLXJl1RNFTDfct64zvCzQqOK89mLVEnta
++XWS00vnhVzuIChu0tP3rkmkfD3fswLGiXrytiBik6uiCoF++dqRKAC/LbaDpm+PR8BCEoIioAXiSROkCk1YxV561R6PzasbWU10jQ/Yjrm
iq5bx8lJLMvxabW9LMACD0vXdlmf0TnJXCAlCQFCPCXbDBG4nI0zfDtkFKohy9I9A1WmYut9/yMQ0I89tPnVSL2u1ve/z730NBAwAkcBCALCQIFIA
IgsXbc/19CRqu2a0o3jWl7pxHEcIwgMACNwDMevPvkVbM49HfWkDB7wnvaKIHGvLQhQ34v0u3f10mtoQClFecxy4JyhUaln0Y+sBHwY31PHC3BXX
PRpdEau/dG7K96gX4VsCU1V9wjD7grHuo1zikQWMBpYRF+pJq+ggVlKHPwYIIqVH8KvVvS8VHu+/shCey+Hxw2Fd8Dr3f9X8CgvQrvncS5hKJ35
V/bbffAC9+0ce1MA5LAVAVOUBSJzjVPOJc4ncljiIjNKAkw40h2+amUHQgWnH8TQTJ5ThBkQSGU4vsWeT6pEBhMnCrhgWUfHU4VWgqQoSnsBxDE
jbbJso+v0qijqV93viDtKcXSHJEtIKwHFSMu5Yf+EQKSUW0G0RZtKY9d9qS12UaWb3ZVlQuh6cR5PET9UKB5VTM/SBZJQ/of/RWWwbyiLDJ0TA
EKnlAEP50k+JFjCARKGgEAIJfLCARK6j6koalr/V+ELiZJuv1/eX3Nch7tdxVJ6ftZxxS0kxSJYwTdiQAgdrsSiRkQ5fZVjiQ1DyDoS5wgKYrEU
ZxmZ/SbCoWhpKQpHlIjKEYI3f1MS0QaCsdJiiIJDLU9IPv2u0D0aRoNvHg13ZFug5nkbjFL7jmtJqaYvknHieq9C8iznV8Apwnkci0tabmyBrkGyK
MIuKPNQCic197bRwHAMSZMkgcB2fgmMJJz5KpSTOPRXjUUDAIzh8Euza+8ZgnESgwEAKQgFToiUpMsRwQ4be12B+PaNEHQrSLKdeBpDUKodphBZ2
ZgoHMuUvPgCxi704N9wRPEIhijWg1K2jJHbXpQ7U5QJL13BvmGMTstNmERRkl2xuwPFC103AkhDwxM4QVIEj100DG6wH0wI0KcJwJd0Q0G0sA7Z
I7EUJTurv709DWBUB6FPengIKw17ji34/I9LScTkqGHeRwQ7FOKTT19wphvWrfWqLxfhf/1XGQk63/AQg8U2dI0jJ/mT/a00kicPKCinAMieMkp
++1Lle2Y1shcK5/ebIehhMUSQAASHuIqNEjqf1/yJy25KkWv/2UvvAMzh0UBRFYcJbC00r2hc509YwJ06me37fJihuG5wbl7sJLANGUAJ3b9i6
070DAEIKiFxFABwitkm47mIT2x0jnn5zh1Mg/yhjM9tY2hy0CHZ5paocV8e6090nsPKXVtz9YxRxcE7WSK78qlh2mIgyAh9qbrbgZiGAaQJAnK
40LMRjKdV3bkChMOM0tA4eurutrcRG5hsQJkiJxzC4S0eBEx91fn4tHrnL0DYU474hngQJLEEHq4pW6DzcTvbb/bx0jmvSMTr6/sBJ0iVce+IPv/0
Y6LCiKWUzqPFM0LewAg3jPwEaYq3PzQ567Iq57k0fW80xxkfJ6E6fK0Ew2uXAoIaigkQ6d0b3kClSaQs6BTQcdQ7L1KHcexG5xKqLnT71KTR6L1
Q6CGG0E7WxGoafIQZmqm0E4WhzvXXXf05lh0ELRVbXvX0pD5BoCGwLMVuwXglFNx+6qAowjKEYq1pb0XBd/aZZzVPV51/aECYqqSBxFEJKNM9S0o
GK9PjAGAA5x1oX9Tz9EWS78bSLD35f1LGUUnSFIY/4e05Sc7j6uEackXiVqY9rsJsBUNic0zjNku/63fdaYntvaynjQoc1vKsxnW2NVvP0zVkv
5x4NZYPWzmWI7FclSers7VQ61mHLZX47RqjL2Zlv3ji+Hv9eLkvVLKHN/MOXBAyYt6rd2abzWaznjYqZ6sEsqeyX5ZjuberJhLz0+wm0lXMXdaro9
81ELN40sp516FJ3HF0uBzldTjxtlgr8sKMaTevl09fVmc0872KtEWSVpOfTmfzC9mF5Xq8n9fIRu2YdvjE9X0lwsrx7JNB3g7PoHxM6W00y3ve2
nxZb/eM7cRLCMJ3R9y20fDRmu62bz0etnLlgN2v/1t3qktdZ0t5s1vMei2zXAq9fM9YU3dkuvFLPG03x+s5XWu75EhW181a7PaRKl1gvpr06z
8qd8cVSn7NLiWGLQ07q9wN0lufN0/WjLXTrlatXG3kbWg6HfT1tFGGcm06X09EBHLxt85aVdhuyLxNm1UEACnXxGzVPM/r7uly4lwc1hIveJryt
Cp73uN3Bb90sj/WY7LUkiFrrYxUtNk0mjZr+2XQmytrbNaTemn/o3Q9bVT2IT7an4qVv3e34WQ75UqX5st7agey9m/V8KJZ3+noiSA0+t03retrYt
7JZTRv7ZrVot+Plh0vZ35yvGts07tbP3rS6t62CQ779FIa595Qz5ldempX8TdeV5gCjd0+K/i5HYuXePzirkYgcdvSca9960ulgl5S0DjtFM
G2vfeuxX7p1m7AVN5N6pcwPF7TzXetN8zLZrMc80edrQW27p0E3B3c75CBWuG802eY8tV/Ks/PstJ86R9K3eakK+YpkybU3Q25bpd7kM9u3QHD
ScrHpfD+rHbLWTg+ULoWau620tSKAwVsTeZzaed5uCXu4Xsm8w1bV9V2prg1Vje0+0jn0j4hXjkmcfNWEZK4nC3mw2y3G9snWu8w69a3496Qxmu
9dXmzuP/NKqDxYXZC9P46Ki10WUzi0ct7YYsKVa5rW3mBwALE1m+eMR12az2Wx2AUku8t16kSkuiA109St/yJYDFqnUx8v1ZrNZDMX9jriCthQ72
Q0uiegV3K0md9BwspX4Pa43jozAFU9xupx4NRvw5TI/2N1ZF0+due/j5cSLAVs67MouWE48lksIP5gv1+v1YlYvImVxMFuulvlpFFFKt12hTwXn
20clzmqmh+ZKGf3FqMxNLOAd/W692lymlAfKbg760sRf86NM2YNI0I25uvNpvN6qVZQ8r1vebCpeYGTg9qxAny+J/L0rsE433wAdWJUUS+zn/bd
ddXRraZKNrpgfEMZ2U7Hsog0VC51TGeX/G9vRSldRXCc6dKAAB53ZiS5k+TZEMH7VezqrP0PVxgQIAQBLfwsPsm90LXyH34pKhJfT+td+
0NRp652Ft12F6U0nQPXyp8B+Sm0NEEQBEHQjxdIoYKmEJfTemn2fitRFPI6TwhB4ihS6ncLF0VUCQ9tK1tiEbgeqXbvXppZuJYLnmSgAIAyDJn67
m5u0iUfHsFJ1lexyA1yuZy/DcXw3Y27dHRMYCoCSHE0mcZwAp35phZS2W0CLcrp0ofeN0+CKLpV/ZVZVFJ5n+pS2/YqZHuH0GTgdMtqE46unJf5P
LULJSLPCFA4EG+P2NGz0V0t4w/dKNMN4tA8/6x6wVK/3ZTwtc4NELAdTvtUwKoikaIAGNF3g3FHIRPj7/J77/19Yj/d/wi01hITIibAlV24
LJLoekZkhY9nIC55N428cthrina0zs2JeTs2fMRZLFWHkz1ToFDn/9KZfTmCJMk8241LKft6gIMUEXv9k9JUVJ3V7XyH0Z34pKhJfT+td+
0pZdkQp2+89GLfBWID0+4Hc2NH7YJhK7hepyDUEXSRMSytcJU5KKpPuLdtxnVQ6fHTAMgslVUPCAtsNC/UXV5wo9vTXrek60T5UMPRScDEchyTaS
3m09iVJkgRBDiCBTLtTjtjhtxw1MMLQdTuFmJYeEuZ2da/EV0ngxl0cgds+0HOXGEuu3gTlsLtoVpMv1BG+0V8UmE3BG7Vt9LjAk7fwr6dm102n40
8/ISFC6J0sf7mNgEphkt6qqouTBFWHAOUmgSM7oa/9BkqRXCyEAmWscGnC1fYLLPq9hDEcmvnf7y9axeBSaRwwnUADABYGC0I4BIImTJAqjNCNWo
wAgsEYfVLdWR2Y0Y//t+kB2DhseiYJ0W4scy8c5hclabISVY13L0xvr25N1Xngxyg+b4fSH7k5LM6KydxLD+zRRfK08prgZuv3YSAAGCXf0fIkS
VLGZ9RJKLFWnk/mtxJ9w9BX8p0aJ8M0w1Psa0co0jGiBhVY15vLkLb8jFJuxXEAkAYQeToihKSQqEpi1M4hgk4IFF0NpKSL7jTH0KeXqf0figSL
xBMLnhgFE0koR/dp9dpYKmcHq12/yIgp1h4/D7Kcoxv3Mjv8JxLNsRxBJcKqSxYHTAxp2qU7TWVomz5+j2C8A37rIxqLNI3t/4FpBno3NW1cc
wiAY0mJwghwau/JU1LAwQkjgFNLQgkChyCKz38/QhJFKUR9VBEBANIYJ3NuDR0n8FXB2D4C8CvH9vvMxc5iYpPyQBxFTKBFvxdZlqJOFXb2dL8hR
hA4eJe/AwAgpGq6BgkAvqEd+wJG0yiza7pdTKodB6Q+cSdP0F1CbzL20KWowUPuxSjo2Q0IwhjSqKsoJgBAkpBE7ha0Nt6XTsADE0jAM55nVG4k
/44ihKM00yQ00mSgNi9y3eX71zyZTSQTBMOBPYmXS2Gwof/rLNUAAAJJHhQ53tnQc9S1l00zt+90z435+KJUR5Kc0vrmCpf6b8BgKiv3b+i4
qfQ0gQPkhlQLGNMzFAogRdFs/c8F0iTC9bAdzzBSMizJMCPXNhtjzSN41z1EUf12yG17u0QVzTJkEw71AzStzxKNbaLltx+2/JTDENjP8LVUYIg0
DSKKLMFvmQ+yqkC3th0rKmbK0hN2nRsv8+gbkgqFACkURT9FFma4qEakCYomRcFEyQBtNjP9BsaRDEUZRmey63/SIIsIo08HfInEnit6Wujbb1A
sUnhK5jASNPvjGTCUnlDN12E0UB1Wgpnxd+7ui/y6a9CUlqgdfuTBgGYQIE+ZstQ+TdLur/T5JmJkm6ZpCYRAFIEdSYbUvRl40cCP60ti6x3ah3
41P7uB+KPPE/NEI+6qKaL5vXBL9FnsVwXLx/zFVmvZQ6ldUqg0dPvdvgeC4YXc6S7wi0vpvFTSUuK5rCwQ2F733p5n6XeIpingVnc1cx3xYGI
CRd6qu/S3i8omm6jPexAGH0u+y21ShrYVwLg+/AHqXedwnKYYmY/yfMYQJ3kquxyBpmEYAUENclIR1WLKJ1F86RegB8mCDACx9Ioik+19LZ1
rp+mI2Zq3KB4V0KIhS59ci9aor39+ZHxvc2PTXxQzGswMmeI19EHQX7vX+RaQnU0vPggPMPqElyHw0IpTuugGkrvZ8ALanfLubR2Eoy/Kc/DnaG
4jCEAj1sSdR00rZ1jvnu/gc45yjqvm61s0Bbf+UJgAoht4nJc1Lw15ILGpIQ78/3kffS110eJnnd4L70od8l3Rf4AQ3N0IE95rKKIwA9f9hovQ4fK
sQcWoIc8ft236bAZ/Twa7PxNL0ZDWAEyhj2TifzEdg+Y/ndnS1PPY38cfyukzh8eiKbDb03NvGgqA3XgD0zVPfBKpaFxbN52YSh1UsQ4u4QZPs
ps4RSFmC/bb4Pnke+SuaAACACW27F9Vz+m1J9fXA0wqARzn/LJ6D3HW7q4/AacPtKb3t91E6W81n1Bt0uyVL1F86RegB8mCDACx9Ioik+19LZ1
sLBDsME7iyoQ0imfteNcDJzsQdKkiS40eW0ThpFMeACdT4JJ0i2k0ZU1wtAvcmpA7zjHDZbwkFwLulrkhVxeja0YV9SLMnzDQ0FT3dzPjsVIYf5K
KNKu0C4Te6nUtuIA7Nveyoe7IQSBU8IzbLVEz5ffuG9Q0XT07/RW7//jefipkGpXcNSnIikFWU4Ku27fqrmfaADAMA1rDb1240FCpnZ2IyCva+6S
7Zfh9TVJSPthraCQWwJ9vUkARPMIFId2zL7KmP7nt9HYbc1+C1E3LZ1r2/70zuY3sxzPwyJ7wYy089N1d5ir3Cp75M7rcw2wVgaUwa776S3cnaM0
+nKgku0NoAp5GnjZ7D9PqLjLmEgDFtdkNNs2za7C2F3f1fEAIDu/cqVD4+ZRL1ZdQs51LdLnV/K7tDn1HedVLL/FYxHLvze9lNqpUrMgU34G3DNkWG
MmQ41ZhdHn1DvME0BZRRuvP9mG20BxjjeEfVpNHAUf5s393Bd+Jp9SuZpFG1w6v/NmG631rH9gj38XDhFmclQ1J9vcoGctMx24s0lYDQ9S3J2u1av
kdIcI4jNMuLgCm6740U5d0Ymr08l6tvqsLwp6l0XymQr8EHLCCGAMAKTSr/2tb0WNEuvyWFA1xvJ+0TAMozptqTKiORknmWjaPEQSaRuG19ZKB7
RHCKwNDGYkrb4u9XiIKYyBziooU/X67U3d2No2iB2Dee72CyNR3i3NwSRNqzzTaLqFuP93fwZMdGEUiI8e27IApLk71fFtp3+Vl06IQz/CkerOW

Ewk7NTViy6i4HmzigiTtPg2sli36wBU6oIgcAJXryBYTWpP18fKru5dmtmqXsXXqTK7Y9EQty1b13PGL7q7btZwAd9urMPEkycNYQkPK3gkUTWKjq
YJcZmZhiCYSARaAaJwiCIAj16qXmogvAbt1Nm/UK01s/LjrPYqN5k2nMomjerIJKYxYlybLNs5cf4sJ32GLwVAQAArN6YPJq25Kc6ybxZzQKR/KznM
BYutSmaKnc0ZD8WE0zacPappC/+nxv2yt3PD/a2LJVQ5he/03LPtODGtdczdz3w4q00bZ326tnQ74fcRcKf80auW4tW9Wbmd/ZKt/czbNoH3Qzm
ikYchk47wZMTlf3221+D0ZknrfJPZqJh+cxD7iq02A0ffnGGpBhDBrYJgwunGhx16fh2DEneURTRUknRhFswaGSMcRmDcrxzn08bE3uptczGLT5
kvjfToLLCaHJNkNBCL357SbDZpCDQMAAKQmndr6QVafTmKTW0TCMKtPomnTa82jJMLNYu/iPIvdjwRusE12Iw5JlEaW0c5HwXch4inyKrKXJEmSw
3rSVpgKAgAAFaY1y17JfJB5ATLnrk9jDvK205xcGB2vMg0kksxOUVGTGh5pZjgKJofj6QalVukBu3K8dxmEjYMaAM5k3hGi1EwkkYbtfTBx09BXCB
vdwp3zCBcn0oV0a27RrFFBBAULPRRnvtDsYi15NYSE/fP1TKtXvYi0CA15Vjwie4C64x/fC3JUK0bLNV0VXKDL7g4qJLYMSfVmlbADXfzq0nspSDrJ
36B0sFsEhute8q5sqsqG0QsgL26fvHi+Er/d4NcIHk0X56cxELN8yWf+McmT+4h2o6kSjzohkwnKaA+a96sAggju3jK2wPU/bzNV0ut0aT6q6IiV
3i/6PvEz77qMvAxw2kd3/cCEPt9VTUBACAMKZJCASKb8LD7I3Ihqlpu6DJ1s+wfbqz+0PUBjgPPA6DgA7X44m84SJJYE98PATgu4VuGsRGdzZB99
EkBTPIEWHg+AH54MrvDRYjiaLByZyGB10DxfdcWTx3SVZa2rIwjk1f3x+r7vA+K8qYggCeD7fgxY4DnDvr0B0I7I6GwBYAEDoEQgqJuPuGLP8wFJn
jdJk0k1EEnisqH15T+lSFPWdUNX730xiqIoDuI4j09/g4FQmul2KADAqq0f9pd4fZb+i9/8mF6hTGNbPt8RrKx+3/Ip3fLMqWZai3BR5VHNdjydt
ja0NkRRVJ0rv7ItYrWkL1LtXjYpp+SOJXdM3zU7RuevdJ3nXe35LxFiig50dZvSvucD4p2BAyVJAjhFGuJvfEBq2595+BSPBFFfF+NUqt2B0+xaq
xjH0WAVpurK6gZr/xlL2qKqG4bGX1cGN487n1lQpIggCTICHw/xknyNCYESQJ74wNAWqzyxW7CT0LBN+MwjgKoxdV+4C4sCIirRoivPci8P0Yp
8k3b4yCc1Tg+T8Gb1f1UQAACG0KRKFC4HwdEv+MpiXZ1Q1DY4kiYkTR82NAHAMURQEAOe/HwB9qqg0AAHFAXH57h6ToAHEcAnDV0ZVRVRcJynUWM0
6gpgutANZUadi0nYGMXla3L61FSNEzRGPoLu98xfmVoQ37VoVYG9wq++dv0impz7pv8uesvWj0AKV+p1T+yTSULeXfoxy330NrFC0KpIDU17EVk+D
VSLAiSu8YZuZ25G0GfYfXfcb/0bd0YECaOIqP5Bpes72gqc5ofq22g89x25m718lg/dsGS1g8hjAyRKA0LNUlWGYGfyn/hUY4xpdnkFEMqqh
SB4w3EAz+pDcVWUug8XNRWfbcfXDWJllyB0pAdI0QD7CqzelDMHQFf/gbHgAilKFvh/iF7t9UerW79HsacA9uXNMMyh9+mRYVexwc3Nj+Kwe4ByF
9zjFGHuillddzck7Gxvm0p8Q53rdPdlea2rs114BgGpAFx3aUfvrde8A1X50Ax/RML3JTxBM0d5EhJOEHjs+09rApyI3usTc008hVKUbsh90bI2R
r8w0oX6LBxH02/17uPLADXC2P2+6eHacNW/ysM0LYK7wkdF3yd+Tyax0eBi1x7m2U51Pva3bdPsq6zDX135xNg15E7c92wVB4EL2v2Pa8vGXcS0/
WGSMBgAHAWBpY1DwtlYPAq8/spx3vJcnNb6ukV1+ranPAQ466srXNqicBCFTT/MXC09wGlyQumhtfc2ZLHGqsvVPshsU+lMyp5K/8jQfU5Kvs
33zVbmg25s+TPkF3J69dj2dt2v6p9NkYwF7f0UFR0nMwP/XZHdngUAMBq4Xb2PtnUwq557BE0KRaQ2E/t0f7EBmvhu+1PQSL2XMKtUqHikbkjXg8qJ
S9WHRoFC+OYLJDSFx7v0pY57PyR7Y5XjnbjcsHG9STi0D46Blhp/m+02Cpf6GaBo053E71uDuRpcAG+E/GDdzdvBkJqffsqwL8VAtt+apjW+ZQ5
+2Fs7H4FWwICFakY9NZxTyLhq67wcXhcZqHsh3bfbKXxj7NYx+qGHqX1cQAAPWp0t2P1hygqW7CJAERYmukAn9RXmx8Q0tOGs1XToZ0PZv9Hq0/p5X
a39amharu+gmV2q/89ms3C3X1LWBBAAQxiYamj3sU6hKJUGUqilW7aWi82uPls92csFcFru0ERA8+7CA/IbDur6cbUIra6odwXty9wgVDEfhydYp
fhe1iU6ziIwirIR4aIm4rLr+Dpht+IdV5cJ8jN7+honoDfmRl0T9vW7tyLsPL54MTPIRugQDicsAa9YcS58WpndT3h6LJYrVrD2WPP16BY2
vA1Jo8b1z7f66kozunBkkr8buV7p+JBichzPscnEiQ0nDCERqf3X5wZNPBjYUFP4MkXYA8D1NBzz05wgcdqvtPv+G7U5ThyhXizxV9Z/j0EJu9G
9mryPx74TgIQpTktY69cglyY18fo0pVngHJfsRRWIHvxzh14XBjED+Zhzb2/RALSQJ4q01MvXQ+ZQEKE7m17/hPQgh8H5AkiQbeyido0h9h4ASBA
9+/+ZoepSgSEskrPw2XZBCEKMqghrLYDNlclt9bELIuYp5j3ZyD6A2N4u06fDMwFyASF1moEkcgHg1VHL1GMhD1x1q0T0dp+jYc1Hl0d/8cujLx
4xStKbV4782DQ63TbjdFvucuiSuD04iF8zwcUfRt/kTQFGiA4SiL2/QCFAAnXEXIikgb+5TcP00hTwvavE87G32gCavqoi0DIUYP0St76ycJuo61
znK0Mb1qi8D0FG3VEyQJ35eng69RKuSy91E25KD7QJ4jgk4W9DkX0qVfCAg+FwT5HGEX1kpBRAYRQhgEAKDfesuDF07FIIRVML6UCucWHVXV
N0h+q4l4+lWEjT2PbiNxnEMCsWZVWR+01fDRf0MVsgQWos+G+/tMB15qbA0nNZSNcCWFEURQFASfpo9jRNou+z+hS0rjPgv3VVM9L119fYaVwKf
vefhzap2Tfjjoiaiv/nV0BfFdNpIkCQoEtURZ3tZdXT7Mq8HQeIAJ/BiJoeTmWAAAIQ3ur1+7NJel7CCpxtTjCuXdxTC2PV+0g14k50jfkATGxs2
7JDXoTPjn0vABRJou/s6S08yAm35kaQJB5772nfsadvrcQDI3UNjgzW+LG65ANzjfgHuuv00+QmxnFEK2zWkjt0PqLShTjAFwhodVe33xdA0ZFTtiw
AKseWKB8T5IigTBeelU1CIIQy9wP8gBw8/wwnxjYuaTtYRACgi7uRk40wLmLoNXwt3aDQMoro0nS00UD14+WuR3L21zlye1TuFBW7wntF3y++v
0ksci1v0FOFV1UedfVn5G5pgsD0MgskiqM4QeDpY0I/n5SD0gA17815oEAAET6xthTog9y9qQmvaWhSrfth4rIs4IEgce1TnDEACnFPwGQSLobkCt
CZD3k2zmKZ7tp0KJN5Ytk9pGgtwnECDzcYHAISb0+5WxhWJjdnP7D888Ryt6bRvd/qnm0Ro8jeSDNxu5uToiikEHSR+oQqevgaoZqWCixNdy7D7
njju17htDxelXyooRji/5YDHHxLwEVNxBBA02e4p2SNRnBezu0vjd37nBdsu9mxrRanZMvK97MQnBLbphAEWL00iJqQnAzsCx1qgsi4XaAgpyntS9
s00a+znHkGirhKLVmGfXBNHAMVxIvY2mxia2P0PJZth59hWFEUJgKABQEHUhljghrXFXxLJgnRuvHU4cJ0hTEWxQAAlABJREFUfCbTbxepUF5TY
Y19VDDXhK8kCg3ypm38TjctRNKqJYdESQh873Y57f3J0KtZfcg08Fzsqiuh2Uw8Z6f3F6GjKtG0geotEoSVDp3+0d1jDBFCotvJqf9F7Q1
aJgCMKFodmoqmauqmkqj8dphIGyqoxvz05pBC5DosAPAEHCN4WhrCpWEVI9Z3wkZI3HEfBNZx/2HeucUz01htskkeC9Vp+CVHUEiynVgG8LlSxnm
4JWxTqC15p4/jhBkysILYvU6QLSt7qmBxFVipy9xBv7nP083pi2z6oyVczkzHK+7J421M1FNz3AtqAxTzniS5eHEwTqbxabGAAQ+sE9c3s7bh0Hy
msy8etuNxDhJxeffC8vxaU2jC3v6YDTzz78gipuilzjh1txQXlFJzdA45xZ9bVZx0dN77cyuNSicVG98U4G7h8cG0SUpiluecoIFgHucL8xd99pXI
BY4ikKpPAQNPQeotKFOmH8U0uyyXte48c8ESeat/ZH+wEDKNGs+ccs37trUi4a+DLbvEAUPad94LpYDFmCV0AABChRnAGgk0wL5aWg08U2P
dRnoKlFMghZ2ewj3HvbM2UxSg4mHWdgdYAE094ruusLiojI78RDnG8ZGfapc43/9H9MekodA1WhNZjvdr0eVAUY0rjXmdof1pJ3/UQMY0xgvD
0mS70c9qV7FKtU6wmmN0TpK9tMmx3CSokGcJ7Smab6B3ViQYlInkdrzw2HeZioIVqlzjUFFPqmA1abD49bTFVUCFa43m+/lyDPsXWUfp1+tlqdlSk
BLHCK1J7jPqaCpBK3KmmCkYUhmMa6a1cY5ylKthPG3UWmap1oTvqcwjGNMZ/vxo3GAzUpPZoFpmuYt8fNKU6BkCLzmVg6jVgAKaJCA5rYl27WVv10
ELRB16T2lnWqWg9EKoIUqlXsm/QFCFV0T3bJ8lH0VDqFaxSRTOCIUJrPcYN5pmkFaZaqVRrdYaTwtNdKkAkufTnbY7hBEVRBE449nI/bdYHrMoIr
Snkm/rddtprCIIGkUQjocGxCK3QfC750ku203eSqAGBvhjt1lqlVLSJjUrdKynW7hZnJ7EZSI5/H4rAcNASG4QURJkS5B45TE0YfXdlN5UWHL18Ie
QzDNDKbiraTTLLEG6SEh/WoIXCCpEGCxzVG61Pq0msNWUdJspu2uFoFq9YTTk4K4LJhmpNtkiRetwUapWTSSyJjNm0lRqGyFVGag+aDFLhmpPdd
qwwRzuR2lKgiPWAq2CVGicNljDjtau3VioHPMJ7mdtRRIkSRIEjmgU3nyfKonVAHGNHqTY+axaDtpnnqpD0bHRMW3unRlXUomVFAPu3mVlidJk
sNy10BqLUqlVmc4oTneQgX4D/NxW6oBpK70xsv9djpocheVQ4ZoDwFa3FPv5u3C3VEKQmtcflFSozqVbBKrU6JzWEGoLVldE6ipY9BrtJAxJtn+0oJ
lNlW0JmwpzUnkTnVxbCeiaJpCisR5xip+BDsp806dulksCzNymE5ajDVdGegpXf0T3GYSEglNwWt15F558Jn4zhMFGWcs5toPW5KXNZAGe0Kk
6p9J6L1uN1gkLWu2U4GjBtNLLJbJatD10oAoDVLVZqqg/z2a57wk13cNbFpnS5KQDGLajptct0tCMoKQDALEXJNnPesoFxiNo4UkRUuWoi+fdCtS1
6CmjNeH1P0BUBXa0x28MRoe0JzPlFL08qJLT6giCFZlPfavSYBXRNMD7eadmSA5nh5iHbT1K83J9Ac/tuLbLJpvyCOY90+Zko/3cUgFUZPNhQkW
DEntY/o6SvZiS+I4v+8cawwUt3Pe+f0bqdzZyfbCGJUSRLtpm2png2boPTm+yuncFhfbKv3epnrnYhgR0L1MrvgeXih7r4gNrhUkgKVbHeCT3PXb
Tj3hbkrTWhckI8WlpbJfI34nVdQjEjJ380GrccxTtcFrg3mBKp1pLmKor0P5vsk2c8aNoOUYve+7HAPzJaIE96bb6LAcNxmVKVXezIMElvt5T
+G4dbwm0EyL2Umb+/9JiHavMs81N5gdhNB+1WU6pJAKsRzY778rHl0RqCvZpNd10iOkUdcx0k40s0q0eeCiet10JQE0UuPwMwP
yRfJEC94wrvecn9RCmzE5eA4JyFEY21r45JY2g+uCW7McSLlChR4mti02Cw+2n0vzzS016+diuSJI3Vlbn0onWPKUqzmeFwPiijxHeP+y7vgxHNG
tXj4sNfLZ008Rax77FPpAX/KeAdPd2PhG+N5D4fL3NXcpgqlS+m718Ph2CM7fcIpc/6ZvD9bSf+DgHd4BB7rjW0As0x7+apfednCCVKlPhoULrZi
c3u23bvQhC70k1BQIvPbe6JPdvo+npf/RayM8RuvxPw562bKEXdT2sZr/pDXza7Dx00L/hu8MW8Fio0757uDE1nU2iNoeuiqngh7G8/rvUNPfxDj
rGShy/k2Pop4FXuCl22v+UQ6f/1nTP3w1wsT9kF13I97K/jyh91Ee38fsXN8GUErUqNcSdZYoUeJbAEP33GFH0zqo2effr300KpqbdwU0N2x1
I88D+R90QM/OKZnjD3LXFgaUdLhauh3o07jSL/CxPwEt8fcFYrmeIStIvQNCkL2JTul0r1ELBKt6x3ZID6bvECdwFasKorql+0fV8PhGwS05o1
tDu0F9i/r6g9FnfFv5Ft9v92m34C0zjX9v/9rz//mff0awf/iSTWjcfSf8r//fP/1TEBP/+n/6g9JcSpT4hoCSf6zyP/NDgvzFV38T+L0f/+LnX
70RR/zsD1g6tLv9v/7bv3X+y1//pFvY3l0GX9YwH3Bf4/+M919hdfupULPgnvchKfDf+v1T8x/qMTyP+h/78QZ/uIN1bnP/z1/+P90z8H6B/wN
PHNW6M6Hx09/Tm0/3TLUoJXu0v3Dj/7BS3/CRX6gPyXP//abfMKKH3Wt4ufkiT52m0oUaJeiRiLSpQoUaJeiRiLlnkL5TWyJeiVKLChRokSJEiVKL
PhuU5iV0Ent2RKRtX3HdWFCMcXmCVR3iW80qjv2XcuBLmaeM985AmWHLxw67rtdLUZr2V5G8mb47tAQ5ZTqL5aEza2uXjTOXi/hwhXVkmUFR1v
nZLPHi/Gwsq8XHkXADP6es8gdL9j8qHem/svkrjqGh90yQUbixrldPpcGUZIoLXsfsz9IvW94UoKHAtxyvN+g34afqIZ8KH8klRuPjoto+loGBh9
U8U/GbfeqxrAigkSn4C79w+qzT5+/JdVE3sV3JmLPYNP2PTzn6k2ZGKXK2Nf0DKiJ4Xb+TeS2cdGRjQ/GQHIQoJRuG+IaKNaRo60w39AkVKRod+
DH3Xww0TzqaPNwU0Vfg9lWR50VZFnmWpnnjnPf2TtGHIPacjsszoiJLiK/TrNg/hZZ3iuA12X2dJ3/4geC1jn3sa7yx+7pIEqzWysRzh14AZ7W+/
q0k/YsXBvXq+d/hytSYERNZmuZ12/8JWV0JjyPaNlBx/hQwkJptWPAkPB4tkfnyr9+wEmWt8GyNemwp3A3WfmgYv0iLMuYLpIsy/JyxwkACBamo
dL4Dz/80Pc3N/X60qA3rHbNEfALJ6lqk0u0p3NwJrPaZ/Hl09b3i1JizzZ25iuaQbka/sMFUNWMB0ybB00FX1p07fjJ+RM0fXo6iia+clnru31DptAFU
PL8DJoikGMBXvc0H4pvkyfcQ+hoJCL/5XDH8cnd8LFIwo8ZiEFvQ0ErxknCn7auuER08EbXZ6qJbdx12Tv8/G8cYyUnsTRZ6mWdW85d03w3P6u
kj+8MMPu0rcBGmqx5E//IBSsjF077wmyNEFY1IULerPhXwXINX0G8KdNxZLcljP1L//QMLdbPZ1j+Xdt5vC9TnEvLY5vNMFetb4y113awVncW8H3
00p47kLevU3HTI3b1aQlw5v59n0KpSsPwj7EYfU2m86y13azuZPcSv36ndgKtKE1hV63a9Iox0Z04fJop0HK47RXnsZuJGm6Ud4qSaN6sVpWzC
uxGgpKKN25iYhZGAnKrbNFE4V3s0LkDQ5TPvSs+WWF0214V0tm9YqLX2124CtM+4te0bwbrtdVbXl7Km7wfbYj72gnDfugL7m4q808i5sANBc
9R7H+nflq0aIkB0J18mEoIwL49a95TeNkmSaDsSKHwMnRug/eysz+2tHUMQcHN4ZLRs1TEEINwI0rSHWaNWP1vI5QDTR9w7T5jNedtcPKI9RVQU
LSdthtK06iBYoKanbjWdH7G9k6wdJg1a/U29LTJZ+OPMOVCCANpowFt1Bu9Q089IHd06n6kt0Zv9A4fisc+4huIVKPLQJLa80dt+PR2PuatTJ8Luh
+FielWhxyyioLchylHwU9ZdGLEfxsKj04dfFXLkxve0YReZ8X6iVgrN2bGSaNoTmnNoDevJaL6DV15clD4aQ2CsvR8LFQwBVfjz8riki/28WU0w1
wd83qwiwksn3r9hJdYz9e5X3wZ0h3d+roL84Rsupb24BXONH0DgS5W0Q3kvr+0LeI3jR14erj1lh/CD4lMHyb9qlct0pUu/wqW7kbfFvsfAiN
j0lrcdFXaPxR0VXJC7C78u82r/drXKNCgwNcYD581I9KRHl5L53ji4r6vpzCUJrtmVdLJhLtnWBgxdMaE873qnU0Kwsoavb7TzXfVd7FvVpQD1f
l9WoS/K5F9zkYniVROhWDM1Y0F27H5ujYPgVf18vAb0Q2Llt46Z1XQGW5YLUXgtcERRUFQRudoIw8nWajYy/QZgQpK8F3LNjWqoHspzsuJPM/z1
J2VBVzS6KhpQE7fFeb+142jVTHFv9E7QKtiVZ78tr3DCV8/UKVZw3H6j1YgP7+dj/jkCXJ7NlX81WN+QRRH7HfPMQfgnSFHfnyLTR6GjI03rhtS4

RURFrW4wVPKEmhjiGKhrw61qg7RvltqlqLHK9PKtWxjWbj81j1kBN/R6Xhh3dtpUoQX/d/90W60HTcETLAUiW043jKJtpmWwFYjZ0k1sRbj5Smh
nC92aQt1avCaJ8kh3lP4THjKtHgE2ZvY07sCjXkqToXJ1RxtsoSxaZt1BFMKbRlOpVDMGqTGM0nfQaQr2CIFhNubvCuZ8P1BoCqowkKYP1ftaTa
ki1M24NGlylt1pwnyXogcYKkKBjX55qTXZiX0qk1DgGkjJUVr1bBEKwq9LKXIbtpS+AESZEEhmnNsv/2nmmlJYGPm615dF01DqCCKSPK5XCUqsfpkM
+1JtEXVULQeSTueYQmKwDG1sXqGP3Tpt/OenA/rcTnThMT69J4173sFlqDziQSem0yS53JkN20LVaTWXc3tYvq+up62tZvYBxmTYRsN29siv2N
1FqFaYxm0+TZDfgkMwKl/t5T+14SVEHmkl79ty77duBzFt+nygHOVXrzem+9zaUbqDcdQwQbXvmk8LXmCNBgOvUiaP7dnhRuYbktE823JpdrJ20v
MBxUnuae8N/WomFqigMu2lbqAKkykLS+gIv2k7SYRI4ThksD+kQ1xGAMc3JNLWSmtCarPdXnUqS9N0hkk375j2YwEBVWwvYx1HdqCILsvttKdwQ
mNw+8INXrTtMQhSb8JWw+4VjbgK/3Jsg5XY/bTFVECFkSSBqVcr1XoqxLSv857CMZYkNb0NoYyC37lFatl2wiSH9ajJbCYiCQZxGF/L6jDvSTUEq
StNhukvt+pNp4Mj7wiD4ztRGGLM2K1Ieod7W54SeLoKuySkvYqYKAIjGCLG59mTdtJk6kgVaHREGoVBMGqXk8HbcVrLZBkEpW5uXfIi5HZa7d
Pw4RhptC4STPaYicVLUSASmVsF0b4pvzaQISwXztD5QRKZA46czFKfb74/NiFP5Df2e9KJaxUqPCEGyaHkJOmkADK0W9zXnAkoisDwHdNKNW75dN
AWGEXsL0VCYcNtL5PbGe2p5WI4aHMNJitJoSHXsyZYFNteHRRs4Dli3eYa03mzCnKv23cDQRqRwICXyMNzO3KueB6gPYjDqkqvV4j1VymNduw
8pVxKp70u0pT3XpreHjLRujR55Q2d2s1RwdhCQx0mKECijofFtFeuLM6ayhjiM7DzCqTdoBQX5b/5Ursfn1nQTvtwKveAbYSe2peoXdTlu365arRC
TDPfstIdyKZ5wcoeRypHqbnLIAFpi4Nlod+p6gSNSW5jmtF/fvLqLWIEGavLFZ90Y2iQodu0WUESOHNhazQYPBEGFyEeSSUD654ZdbMBViHblwM
vPWwLwM6f9P0vL7spjK9S0PFWHT0NU+d666T1V768E9u9iNtZHXkjApjYbCve+uxn74C4jeHpbj1LMBSF0ZLQ+H9bjJVLNmeLMHbJ/S/550+wc
bjsVQHGPV7TP9azn4+aAie1b3dZFWFUE4UbzeECamqt8zJNgXgtyfrm8Wb1YU00/67o4j1AWEKTe2xZpL3zyNW9WEa41aELMFU0wmyR6uP59cfk7H
wvY0eMsB4N5kiTJLRczYdurI1VpMftv573Wal+fSJVMBtG8VTt6t+2oNz0kxzhSRs7VTBEGG33URTtpo0aUpFG6/0hivazZg2ptu4vXqebSY69m
yoIwrQmy+123GzP07ZGSTZ700p126sj9f8EVCVtB1WSdqfKZK5didrJnRs13HmN7RWGBN5jTKhFtrJofr7bTVmkb5LWLXVvXom6PtSDgy11U7i
8Q8kSpvZrpL6KQK2ZEka76J0U2L1290qPYZWGsEnPzFXVY/naqp4v0k9j9tKnoSpymgi5MGPZq3h3he0nvfHuJLITNUAHCTvgsFrruJ9s2
W6MrmwezduKNNK7B1u3ZqKzCnE17uteHcvknKTrdh2rpySYm80WqCgch4L0MDnftmuV7PL9WMCmbGvexSVaaHjFopLNaRZy47LWHbqyOV2
10Zh3rmrhgCAVOpSazDNbT67U3Sn0etJW+KE5nh5070bov1UqSIAIFVgaY1muYEqLto03jijZTbuZHp2mLfqSLUx06S7cLHsz/xwMajWxCo2mjWqR
+PZ9qC7ppatKsL0lvtDF03nrTqCcL3l7hBF+2WbQbCjeUJJY3+50WjerCLCYHTikuSwbNWRsUNwPaN1j8EAAfHNaPYmuRH4RGs6jAwkokx2hyg6b
MdSbak1Jtv9IYp2Y+UUVsA0H2J2u2sTYeQZzXqjdaFwd1M005LMZYxTYCYw3JnEPmQNG+eHpAc5s0axo12UcA/2st9c3ZcqBK/wuq1XHGdGzteMB
Ff0xybRb9C9UIutUteFMYavir/ge7sVAt9x0LUpG07LZsvY+YXeazVmt2/DeAcNuL1FmJyVwvT40NSWJli2u0Yv244JJ7LqndpD8A01HFFJZtPdR1
v0nYFrh1549nS5XY+avXXHeOXGKB+o5KxvU+CwrM+HqYJ1b2Qgm7abJ8bqwuSJECkU0MuJyWsyDF0LNXmsi8RtZPSF/lionsft47nuZvTDdV56b
Le072CLVUNUMImFUE2BLy6IZF4YoCR5FKmeo5xo3qwiQvraAW5KsJgEhRNdKwreTgt931PfdQRe4BhctE5bLotJEs4nUA7JGQ7M0q7DxWsdC2Y95
vTj0mM+R0FhGaNakXIgs/kMGmksCor3zDduLLiH3dqyP1o2d6YjtxLPwriiF3YwHdHFeUSXbjRvMYIN20L9Tk212TvtvVwAAWJvrtMTFm4Lzt
e3mg4Abk3PtcxwW5QGAyegpoFKiLEAELsZJrSjRjtonkrNk4tDP+u6RLdRoFs1QauvTCXvmbPbDZtBjNejPvbieMwYIONFvIAAGsYPPwqgqFmuI7Q
x11LRfWmosCApCWepKvu40BAJTKVLAESAEBJAo/D4lHnAcVJikBURLJRVGo1RgiZFCVEmuAd4MMvE5QSVZaiLMLPP21FAQCAIAgohqqUULEUQBQI
mepOAHCAOIwDMPNJgCnZsYL0/JoXct27aC8Y2x2B5KizN0UP0zLkAvvNN2JGfJaR8Y3tvtS0hjFhRqogWvXZgFSpee3wAkSBQDgvEyjYfB4XT67n
hBFgGT56+NN31jIw0722bTvBwDgxGmjSxgGoe/5MQCAUDsa5NN92CBuLHNFajqfSYPV05cbD8JFp+Nppl64/wmC51TujItxX1mWR6lyul0EpTW7
izr+JP+seEBF8BDKdnwq211B0DQZLB6Y+AADQx2tDAHCpod/qBcW/vE17jIPT9a83G+eFqM+ZpV0j+8t/9239F0tpfp92dovim6/potW+batHxtes9K
zdfHgxvmaVp02TPj0L//sf/5XJGu4R60pLqKMLA7bN+HnhkKzhs6Gz2Kgaeba1wRdvZdEHUEFC9sJadW9k2Jp3F8ANvgB6gmEwKk8A1KBiH
EUJjvX+MgPhaSRbtZBAAXJnJZPAxvNqQ104vNB9wknwepXf/Gn/4akZ060TevjrgKtLSBJHJUZSYZrREOEKRJQ5SskID0A+KNB9JkmeRwKk8AFT6X
SSce00uYppNzY2kKSLwPP+OmZyx6rBUCt1ZdCX0T1qGZLC/JdV4Ydk+rWY71nBeE/GV5fgQJn8SRRU+QgE5XzBSgXKiYsegsyoIEPohALFr2gGrP
fg6BXLjseRGLQPXcmJee8ZKf/RMLdv6eswbF4bhSfb3m0S9Ogd+bZorkG0fNwX8LY48H1A4PeGBsUJ4uzDzL3EaVLmKVo3090bX+eEz7VXJCS9A
ACAszwRhheHQp6FaK12bi04zjUwtJRS+Z1Nywwq7TD0AF8L9/+Z+X/u0VHFVEFRFNv+LoXw3iHFIgJz9ghnz+dF3sHl01vAs+8Eeq1dzzxberZ
ACAM0DLSJC5DRdqQ+CXf350shCtVxWwnRYUf7v1fR8ACEAK8kxzh2SAWsdj1pUGuXAgAAa0PwIa+rmSRxsANT4IKY41WYAhZq92MDlaot6Q27
NKrjJtHpdDkvq7fUJjJM+ASLGAAB53ZyUfYVHPft//l//iKLE/nFvc1Xo5yJG5bVJR1D2sCv4aWpUJVer/8Mx7QCKw41RXSWvhPRC+BduzK711ztj
iMudNLHG6R1loghUYEpt7tbSsqGPNT+iFwIqq4bevHXGB18LwCx31dVewAAwAi0zsCd9i1VjG0o8EqBPC930BiCFAA3q0osecM+/YG4DjqbwBgY
Y87yguXda0vYjLfmrKuG7K44HvhzhJnFqJUtTNl06+HFCsIoHjBQA89ZmIv/EBoZ6fQVCQ23LyeYwr9xd7ttbvrnrjn9084DH9EyfnLkV5T4p/Rl
CWrmFQ700gQY9jwfK0T5WvIsngqzK1h0mGm+3fwqsgp9NH3fmc8X40gCedV7fgxyTFwkos/V/6Nv6eoCBQDEIUW2D20z3qjt3l48JorH6IoC
KAchwBc2Zv0YR1L7p1+a3aMzL/p0s+7GknGYFHxf/yff0xrvre2mB5JXW7/YXfT7sjrncLPXgRTet9w+2HnmMaRveXmsF7dkYnd4o+BCRJorHvh
WB4HiBhb04eUkf0+ZWhv5pqAwBAHDUUy7zZEsnu3qCNK5Q+GkaKRqmaZ9hd3vGL8ytCG/6lyb9JexpstWQjUfYm6sbrD2n7GkLaq6YwG8iRYJB
82RUgXqFL9nJiEw/ZXxBwAA4Gr8pr04EQ60H0dSDXw/xknypYESQJ7400Y/EKJFVUIIns7oWUT0BcoZbqz+0PUBjgPPS51E4PxsTPmPmgy5EdYYA
DKvQmrXAQgUCKVb6SDEjm6kLeaA4ALczIBhIVrc7f2qzbpp+LLcVWLBAqAotetckrjHAMXF98nvu8cr8PwfA7er+igAAIKQxRAIA+IUU9Dp0/SPeJsGah
unGogox+qxqF0Qv0dOif1b+GN933REAENP6//TFD/0Q6aeEiYELAALQ78Y2Re9y3VP0VLTYHKR5W5QmKs+01yEKI7GmyJ1RyTA/lpMdhnd3/eBy
JmK1E8ecgYUqQ7DxafAGvL13uAKM/X45LQn5Sg+v1jQ8wJ3AMb1T7azWG8pY91x2e5q6PXvqgmDmPREDZQ7Df6kNeHwcbuX8Gyv/0Lt8r6vQ9fos/
Re/SW0dUaaxLZ87ubH7fdPDteGqfXPFxehvK7WfjU0bV/TvaFLG8ar8V7GKKHxjEg6P7C105FxG922Xncn8QCURU9Vrdt0+yrrN1frYyHtoqLX
XuYcWg9oaxa8mLVoVgWMMynVyjfiLVX1lbGzGqTADjqzVrcTVNj8a5

7HC1Bl0kCSBYzilPc2yE9+aVSaRdg3meZ6RPwZApC6dwNSrWB12csfbcJiPWLINAVhdarYHec9zxzsUHBEDDSZvGelQEMkcnhugS9yLVPfTRh1Ds
GpdaI3aHIIxjffQVWBK0JbfxmzpuZFnRY2209S866bbqNj3ndt57wiVtMVIPc412m2pILS55mR7hyThfAInn6swFWIdV+FikzhmLaaS+oRWr1FHk
kxjvPYuPeahkIIKw60kWo+bQ0lyUohlVDB9GLEIE9Vur60bKr6riP2wHChZhWfqiNITRrMD8L2JFQvsh6fkQ/nbvQ22k7bUq2ehejbSZPBAMY0R
vM9ZHzhJn+PjXfTlsIxDCeLLC0x4801HoJ22mvKVQrjJI9N1q2LYkyd9NeS6ohAKLJRVS8BeHfYTLuS3UEYHWL1btXJLdtvXXZp3LWb7qNDsssd
OvdDyEenBNb4kmk6cM/Uq8+HYdZo3rM+P6tI5oqzMd5ZymLeZzhXE452X//cXlmVAlvm/spw2pd5qkRLURgGEPjxN7Bd8hqZZ4h02AgZ1g9HuLd
bteeXR6ZomfAj7H930pkvzgcPHTw603Yz8SPpmfSpP/WdXI7PRV4Q1F1YtSH0D54cq8/+3Nt4Fy1VlJAEkMqdvux+cuegK42B+yfNf0FLi8YGLi
n3IvWoI03XfT7e42gmDR79j00HxHxqQHCFdmxwTdu1/ivxPxqj/0ZDN6W1L3nxQ+MkFHia+GVb+zoezTt0ooSVM4GpbqXEuKN3s2FrXVa0POM3mu
0dgdR2i69x3gCUK8e2FVfdx3/d9yxHvB4aLnxhuvVcflfTWcLD8+0+Bb1BafIjW95EksZ7sZ44rrvAqhobj47LfOrIGTLCXVDs4am9spZrm96L
mZvtLfddipLq0H50+S7TQPBdW/88VglXQ70bdxzzziGyJUp8V0BREAbBKcuibw8dVLU+8T1QiZ8EULrjDjua1kHN/p0pd3+i8B3DcHnb1r+9ydb3g
m8xrHo7vu2I9+PCxc8L296rDyilW9bzhwK9jNLkPwH/otvtfu02fPcIV8P/7T86v/nv/xT888+pP/7Xv/ja7XkeP/sFLf8JFfqA/Jc//9ptKQSKv
/NgwSee8ItP7X7w3+M//n0d/Y4U43Pg2ca/t3/t+f/8zz8j2D/8RF9R4vNB/OEfom6/a/3t3zr/5a//8i+dgP8Pf939uLc03y+plngELPxjlf+ZH
xLkL77lt+ifjNAP503P5ZIGfy/wKb7vC5PkB4WLnxluvQM/x/HPZKT55D8DPYRJ8rXbUKJEiRILSpQoUaJEiRILSjYF+0fslChRokSJEiVKLChRo
kSJEt8QyKlsIRILSpT4Sog3lqGqsqha3+zHbeHGSnKHGSYbu6/S0Cpa7zLS+L2Nstsg2/e0uQxcy/E+OpXah3f/W5BniRILSpT43LBOYkuUKFGix
JdBvDAogh9655+qSfZtu80GIQCbPkVQndX7HvEhLRwReSaQd1Gevvy0CaXVjvGVs7ThtNR2UdtIHg66Mq686GTww/v/rcgzxILSpQ08b2hnMSWK
FGiRInPQ+y5Cz/7GyVomQZ0ycY3jhuQJAFQ3jBYAAiKpmnqDbMZ33VPC7lvruQWoavLQ7JvqrBcHI/Sc4Qbd3VvlfSLAKV10wAdsBp66PXYD890U
qY7KVGiRiKs6CcXJY0UaJEiU+DbxmGfZZeAKqzXVM+HrAQBiEA6Gn6Qsima2kvnz8Q0h3d0u2afWMLt4hXXcOhjA79pumVZ+rdxbewPRYX0zpqG
uY3u127RIK5JUqUeAPKSwyJEiVKLHg04WoolYhKq4bKUGSK4tT5Y9ZwY2oiL6qqzL08ZQ5CAIDvdPT+3/luV5Y1y4s3dkckUdJYAQCcxVdvr373u
1VXlmXN80Dn9GUKxTX3+DDf7ag8L6uqLLJ0dor9osPzoqqqIKuLHTdIq+k4v131VVKdrM4qiT3bEHLRVmWR59X+IgAAxBtTpXGUKnWVpwgcXUmf
7tSGS9M26dFEb/ovKnxLC+rqqb1zxPU0NF5Xk7bJA9XYfpQffhrz9JL2B9ADZDmRdLVZV5mtfPM/pr4UIEeE/gFzdaKoX+gLPacBEAEG/sjkiRY
t9N20iKMrkxrQ30gTwwqrLI8prlxQ/GF979fI2LvsrzsrzrGg4/vsrlFGiRiKsJQqQLChRokSJEK9i3qwi3GB7SJLksGzVkuPjniRJcpqglWpj
kiSJEEn2E6LSUaaHJEk0EwmpNufHu/dJAam2ltmvqYJhyuxc9IRBSbEb601q26xjT20bpr1mz0Y2SJJr1BuskSZJk26sjzGCBjEKya2CIMo1glcybt
Yow3idJkiSHeb0GcaPd8e56e36IKiTaDjgEKyaHq36u23Wkcm54sp8oLYqU1ZUsW1Ukq3g/6Y132d0QIDA+JEmS7AYMUu9ts3uXg8E8SpIk2Y+4i
+7nUCTAItoGnBcd278cChnCjRHXRVcN0lqeQo6mCnAtP/50lqtXsGdveqbkFjyvs/gX2E6LSay2jJEmieauGpVe8o8ISJUQKFGiCOVKbIkSJUqUe
AkoQeAAAJyVWTWmgXCA0DGdmFez9DyErPGxYzpv/yY0XpiWR+sale7LRXnDoFAULFj0AAAAHCAKf//wHxwrJ9Ws12L+08Juryzl/n0viKAAox
bNUHARXVcWB7fWMAcP3D8q7LxLymEuAahNRyG0FIAwhbWInG0cBAIAgiNMV8QUCfBw4NeNkDUeXdlp5+KV7fMyfSpEcZwAvpfdAY4XthPQarrST
LE07i1Wx9XQ28cVd/+M0LVcmMssCgBAWZakV+5xgfttFZYoUaJEiRKf+NnXbkCJEiVKLPhukX0w6nv/f3t3C6U4EqGAuE6LXdoRL3aJIw5ccMQFB
464wYEL53DgwIEbHHHETRY8M80dIacmbtrRbuKmV+UEAwTdsV0373ZmX300eBRdjaqX6mo0mqUdjo8SxjQIEw7A3hdwCJ5Sph1LHGKabN8405pEH
CiFJAF44eSr4FxsDhwjBRpjEMQXiYJ5U6BkKiUQejwPy2M0zMub8JRuZoPJ0iWUyDi/J1Ym4WQUxEAp4TFAFQAgGVWtP//9BgAASnMp+0UbaD0LV
nBCLzqeQ1tByDs9BmIp3e6N4S0bzI975h00ZFaj14rAACQJNz68nD/luY/ncESD5qNwAAQCKqcnM4oFYrwmIEEIIIFcIiFiGE0Cd0eUw16jX6c
pQELQpi7gSjN4eRr56CSy5WlHMRSDH3nIkWxvMagWS0CPLUJtBw9v04nWHAyst+f72H+YfM9pTXUk0E9KXPgPEHc+hjSDkHStInf6z+j4/cVLtB
4Hzuvj56V+uYhBMqj8xIEIIIZQL24kRQgh9ELNMEIEicj7JuQBMvMx7P6hKXZJsw5YHseCVV+8HfUsShrSKCzesChCKYGTp5caY8BjflFYJ5tYm
rUfTTAWyZrLvXsZ4lmWtUnFqM/YQNjZwtocRDMg7TmnH9RSqLQer5vLDein/cYfYpK/xQzNRD8hRbvtwRECCGECMERixBC6IOo4zXIOoj2FYwI5
2vS6DTo/gKd90LE6YtIrdNi8aQ3iw/flBIopca3mxQARPK9dZcQAKLkdPKSmtdicRduP0qjeZjWvMYrC0NmmqdRmdOwZDSaxXK/m00NPoxCso4lg
BRJckyTEAIPfXIAKNWII6M0AGmcFA3hLdzAlyM1r6H9PRG151m0QRDPURVMqq0GjWeD4FBIyh8VLExp6fQqpFoNIg00X40IEIIIVTsV0+WQggH9
EU8bhftigqG01zePe5up/sX49Uuy7KH7bRdt+tu063bdnu6fRr3u1teGaqqV+rd5T83425D6B7XSHi62u7uFzxcURa9f+de3u2y3uvZdQ1EM17++3
WVZ9ni/9N2yXirPrLsBw5XD9nDyq+UFLVUttvjoVtSDX8d+5A9rvyKqpbKdv6r4sgd4ur46Ka+4Vmd8uuXYKS3V1sHx62i65dAr3u39yf53rbN
ZTK+0TN+2XXNkqqlRst2nrSqlytbjLdst2WVUvVzvTn1bUsvt6+1jlt1fu7qqGhXXX+2247quKCXD8g7HbUPR6/5N3hzevA0s2vBKuxg2ywoYr
j9dfQ+29Q8zgc8y8Y3DJ0fzh7ydtiv6PqP9Bv/o/81P/9xuNXTLulrSyxw73p5uHz8aECGEEMr3R5Zlv7q0RgghhD6ZZFKtht7+L0sXkQZeByZB6
+wJbt3nLgXRX50AKYIIFtBwHziHBBC6ILZmfXlBbC9/6Kg/1saReTtp0pwjMR+E2mCCFSxCKHfChaxCCGE0CVi9a0JNvP666KTrJ/EejZPANKoH
5jn52h520tEtSDovHAZEUIIIIFTYDsXqgghVEis12m1lnfn6Wex7pLeBJrVm807p40S03jNwC1603gohBBC6Cv4D/L6p0qCLSRDAAAAAEFTkSuQ
mCC" style="max-width:100%"/></div>

Inside `call\_events.py` – `handle\_transcript`

```
```python
async def handle_transcript(payload):
 call_id = payload["call"]["id"]
 transcript = payload["call"]["transcript"]

 # Get the last message from the transcript
 if not transcript:
 return

 last_msg = transcript[-1]
 role = last_msg["role"] # "user" or "assistant"
 content = last_msg["message"] # The actual text

 # Find the call log to get business info
 call_log = await storage.get_call_log_by_vapi_id(call_id)
 business = await storage.get_business_by_uuid(call_log["uuid"])

 # Broadcast via WebSocket (immediate, no throttle)
 await ws_broadcaster.broadcast(call_id, {
 "type": "transcript",
 "role": role,
 "content": content,
 "seq": len(transcript),
 "timestamp": datetime.now().isoformat()
 })

 # Update call log in storage
 await storage.update_call_log(call_log["call_id"], {
 "transcript": transcript # Store full transcript array
 })

 # Slack (throttled – check last update time)
 if time_since_last_slack_update() > 1.5: # seconds
 await slack_service.stream_transcript_chunk(
 slack_ts=call_log.get("slack_ts"),
 role=role,
 text=content
)
```



```
FCM (throttled – check last update time)
if time_since_last_fcm_update() > 1.5: # seconds
 await fcm_service.send_to_user(
 uid=business["owner_uid"],
 data={"type": "transcript_update", "call_id": call_id, "text": content}
)
```

## End-of-Call Processing

```
async def handle_end_of_call_report(payload):
 call_id = payload["call"]["id"]
 transcript = payload["call"]["transcript"]
 duration = payload["call"]["duration"] # seconds
 ended_reason = payload["call"]["endedReason"]

 # 1. Summarize with Groq
 summary = await groq_service.summarize_transcript(transcript)

 # 2. Classify outcome
 outcome = classify_outcome(summary, ended_reason)
 # "resolved" | "needs_followup" | "escalated" | "incomplete"

 # 3. Update call log
 await storage.update_call_log(call_id, {
 "status": "completed",
 "duration": duration,
 "ended_reason": ended_reason,
 "summary": summary,
 "outcome": outcome,
 "completed_at": datetime.now().isoformat()
 })

 # 4. Deduct credits
 credits_used = calculate_credits(duration)
 await storage.deduct_credits(business_uuid, credits_used)

 # 5. Close WebSocket session
 await ws_broadcaster.close_session(call_id)

 # 6. FCM: call ended notification
 await fcm_service.send_to_user(uid=owner_uid, data={
 "type": "call_ended",
 "call_id": call_id,
 "duration": duration,
 "outcome": outcome,
 "summary": summary
 })

 # 7. Slack: final summary with action buttons
 await slack_service.mark_call_completed(
 slack_ts=call_log["slack_ts"],
 duration=duration,
 outcome=outcome,
 summary=summary
)
```

## Groq Summarization

**Model choice:** Groq llama-3.1-8b-instant over Gemini or GPT-4o-mini.

### Why Groq?

```
Groq: <500ms latency, free tier (14,400 RPD), LPU hardware
Gemini: ~1-2s latency, free tier (1,500 RPD), Google Cloud needed
GPT-4o-mini: ~1s latency, $0.15/1M tokens – costs money
```

Latency matters here because summarization happens AFTER the call ends. The user is waiting for the summary to appear in the app. Under 500ms feels instant; over 2s feels slow.

### Summarization Prompt Strategy:

```
SUMMARY_PROMPT = """
Summarize this business call transcript in 3-4 sentences.
Focus on: caller's request, resolution (or required follow-up),
any action items.

Transcript:
{transcript}
```

Summary:

"""

**Why not structured JSON output?** The summary is shown to the business owner in Slack + app. Natural language is more readable than JSON for a human reading a notification. The AI call classification (sales/inquiry/support/spam) is a simple enum extracted separately via regex on the summary text.

```
groq_service.py
async def summarize_transcript(transcript):
 """
 Takes full transcript array, produces concise summary.
 Model: llama-3.3-70b-versatile via Groq
 """
 # Format transcript for the LLM
 formatted = "\n".join([
 f"{msg['role']}: {msg['message']}"
 for msg in transcript
])

 prompt = f"""Summarize this customer call in 2-3 sentences:
 - What did the customer want?
 - Was it resolved?
 - Any action needed?

 Transcript:
 {formatted}"""

 response = await groq_client.chat.completions.create(
 model="llama-3.3-70b-versatile",
 messages=[{"role": "user", "content": prompt}],
 max_tokens=200
)

 return response.choices[0].message.content
```

## Owner Escalation Flow

sequenceDiagram participant C as Customer participant V as VAPI participant OF as owner\_flow.py participant SS as slack\_service.py participant VS as vapi\_service.py participant O as Owner  
Note over V: AI decides it needs human help V->>OF: tool-call: notify\_owner {reason: "..."} OF->>OF: Load business config OF->>OF: Check escalation mode (slack/call/both) alt Slack Mode  
OF->>SS: send\_owner\_approval\_request(slack\_channel, call\_id, reason) SS->>O: Slack message with buttons [Take Over] [Decline] Note over O: Customer hears hold music O->>OF: Button  
click: "Take Over" OF->>VS: transfer\_call(call\_id, owner\_sip\_uri) VS->>V: SIP transfer V->>C: Connected to owner else Call Mode OF->>VS: create\_call(owner\_phone, caller context) VS->>V:  
Outbound call to owner O->>V: Owner picks up Note over V: Owner is bridged to customer else Both Mode OF->>SS: Try Slack first Note over OF: Wait N seconds OF->>VS: Fallback to call if  
no response end

### Inside `owner_flow.py`

```
async def handle_notify_owner(payload):
 tool_call = payload["message"]["toolCall"]
 reason = tool_call["arguments"]["reason"]
 call_id = payload["call"]["id"]
 call_log = await storage.get_call_log_by_vapi_id(call_id)
 business = await storage.get_business_by_uuid(call_log["uuid"])

 escalation_mode = business.get("escalation_mode", "slack")

 if escalation_mode in ("slack", "both"):
 # Send Slack approval request with interactive buttons
 slack_ts = await slack_service.send_owner_approval_request(
 channel=business["slack_channel_id"],
 call_id=call_id,
 reason=reason,
 caller_phone=call_log["customer_phone"]
)
 # Store slack_ts for later reference
 await storage.update_call_log(call_log["call_id"], {
 "slack_approval_ts": slack_ts
 })

 if escalation_mode == "call":
 # Direct call to owner
 await vapi_service.create_call(
 to=business["owner_no"],
 from_=business["phone"],
 assistant_id=business["vapi_assistant_id"],
 metadata={"type": "owner_escalation", "original_call_id": call_id}
)

 # Tell VAPI to play hold music while waiting
 return {"result": "Owner notified. Playing hold music."}
```

## Owner Decision Handling

```
async def handle_owner_decision(payload):
 tool_call = payload["message"]["toolCall"]
 decision = tool_call["arguments"]["decision"] # "yes" or "no"
 call_id = payload["call"]["id"]
 call_log = await storage.get_call_log_by_vapi_id(call_id)
 business = await storage.get_business_by_uuid(call_log["uuid"])

 if decision == "yes":
 # Transfer call to owner's phone
 owner_sip = f"sip:{business['owner_no']}@vapi.ai"
 await vapi_service.transfer_call(call_id, owner_sip)

 await storage.update_call_log(call_log["call_id"], {
 "outcome": "escalated_to_owner"
 })

 elif decision == "no":
 # Decline – play polite message
 await vapi_service.send_message(call_id, {
 "message": "I'm sorry, I wasn't able to reach anyone right now. "
 "Can I take a message and have someone call you back?"
 })

 await storage.update_call_log(call_log["call_id"], {
 "outcome": "declined_by_owner"
 })
```

## WebSocket Live Call (`ws_live_call.py`)

sequenceDiagram participant M as Mobile App participant WS as ws\_live\_call.py participant CL as LiveCallBroadcaster participant CE as call\_events.py M->>WS: Connect /ws/live-call/{callLogId} WS->>WS: Validate Firebase ID token WS->>CL: register(callLogId, websocket) CL->>M: Send buffered messages (last 200) loop Live transcript CE->>CL: broadcast(callLogId, message) CL->>M: WebSocket push {type, role, content, seq} end M->>WS: Disconnect WS->>CL: unregister(callLogId, websocket)

## LiveCallBroadcaster Class

```
class LiveCallBroadcaster:
 def __init__(self):
 self.connections = {} # call_id → set of WebSocket connections
 self.buffers = {} # call_id → deque of last 200 messages

 async def register(self, call_id, websocket):
 if call_id not in self.connections:
 self.connections[call_id] = set()
 self.buffers[call_id] = deque(maxlen=200)

 self.connections[call_id].add(websocket)

 # Send buffered history for catch-up
 for msg in self.buffers[call_id]:
 await websocket.send_json(msg)

 async def broadcast(self, call_id, message):
 # Store in buffer
 if call_id in self.buffers:
 self.buffers[call_id].append(message)

 # Send to all connected clients
 if call_id in self.connections:
 dead = set()
 for ws in self.connections[call_id]:
 try:
 await ws.send_json(message)
 except:
 dead.add(ws)
 self.connections[call_id] -= dead

 async def close_session(self, call_id):
 # Send close message, then clean up
 if call_id in self.connections:
 for ws in self.connections[call_id]:
 await ws.send_json({"type": "call_ended"})

 self.connections.pop(call_id, None)
 self.buffers.pop(call_id, None)
```



## Client-Side Reconnection ( `useLiveCallSocket.js` )

```
// useLiveCallSocket.js
const useLiveCallSocket = (callLogId) => {
 const [transcripts, setTranscripts] = useState([]);
 const [status, setStatus] = useState('connecting');

 useEffect(() => {
 let ws;
 let retryDelay = 1000;

 const connect = () => {
 ws = new WebSocket(`ws://host/ws/live-call/${callLogId}`);

 ws.onopen = () => {
 setStatus('connected');
 retryDelay = 1000; // Reset on success
 };

 ws.onmessage = (event) => {
 const data = JSON.parse(event.data);
 if (data.type === 'transcript') {
 setTranscripts(prev => [...prev, data]);
 } else if (data.type === 'call_ended') {
 setStatus('completed');
 }
 };

 ws.onclose = () => {
 setStatus('disconnected');
 // Exponential backoff reconnect
 setTimeout(connect, retryDelay);
 retryDelay = Math.min(retryDelay * 2, 30000);
 };
 };

 connect();
 return () => ws?.close();
 }, [callLogId]);

 return { transcripts, status };
};
```

## Mobile App Architecture

### App.js — Root Component

```
// App.js - The root
const App = () => {
 // 1. FCM handler (always listening)
 useEffect(() => {
 const unsubscribe = messaging().onMessage(async (remoteMessage) => {
 const { type, call_id, ...data } = remoteMessage.data;

 if (type === 'incoming_call') {
 // Show overlay: "Incoming call from +91..."
 setActiveCall({ call_id, ...data });
 }

 if (type === 'transcript_update') {
 // Update live transcript display
 addTranscript(data);
 }

 if (type === 'call_ended') {
 // Show summary notification
 showCallSummary(data);
 setActiveCall(null);
 }
 });

 return unsubscribe;
 }, []);

 // 2. Live call overlay (when active)
 // Shows: transcript, whisper input, end call button, transfer button

 // 3. Navigation: Stack + Bottom Tabs
```

```
// Tabs: Feed, Search, Schedule, Profile
}
```

## FeedScreen.js — Call Log Display

```
// FeedScreen.js
const FeedScreen = () => {
 const [calls, setCalls] = useState([]);
 const [filter, setFilter] = useState('all'); // all | calls | searches | chats

 useEffect(() => {
 // Fetch from backend
 api.getCallLogs(profile.uuid).then(setCalls);
 }, []);

 // Merge backend call logs with local notifications
 const mergedFeed = mergeByTimestamp(calls, notifications);

 // Filter logic
 const filtered = mergedFeed.filter(item => {
 if (filter === 'calls') return item.type === 'inbound' || item.type === 'outbound';
 if (filter === 'searches') return item.type === 'search';
 return true;
 });

 return (
 <FlatList
 data={filtered}
 renderItem={({ item }) => (
 <CallCard
 caller={item.customer_phone}
 duration={item.duration}
 outcome={item.outcome}
 summary={item.summary}
 onPress={() => navigation.navigate('CallDetail', { id: item.id })}
 actions={[
 { label: 'Call Back', onPress: () => triggerCallback(item) },
 { label: 'View Transcript', onPress: () => viewTranscript(item) },
]}
 />
)}
 />
);
};
```

## LiveInboundScreen.js — Real-Time Call Console

```
// LiveInboundScreen.js
const LiveInboundScreen = ({ callId }) => {
 const { transcripts, status } = useLiveCallSocket(callId);
 const [whisperText, setWhisperText] = useState('');

 // Controls
 const handleWhisper = () => {
 api.sendWhisper(callId, whisperText);
 setWhisperText('');
 };

 const handleEndCall = () => {
 api.endCall(callId);
 };

 const handleTransferToOwner = () => {
 api.transferToOwner(callId);
 };

 return (
 <View>
 </* Live transcript */>
 <FlatList
 data={transcripts}
 renderItem={({ item }) => (
 <TranscriptLine
 role={item.role} // "user" or "assistant"
 content={item.content}
 timestamp={item.timestamp}
 />
)}
 />
 />
);
};
```

```

 /* Controls */
 <TextInput value={whisperText} onChangeText={setWhisperText} placeholder="Whisper..." />
 <Button title="Send Whisper" onPress={handleWhisper} />
 <Button title="End Call" onPress={handleEndCall} />
 <Button title="Transfer to Owner" onPress={handleTransferToOwner} />
 </View>
);
};

```

## CallForwardingScreen.js — Dual-SIM MMI Codes

```

// CallForwardingScreen.js
const CallForwardingScreen = () => {
 const { simInfo } = useSimInfo(); // Kotlin native module

 // MNC-based carrier classification
 const carrier = classifyCarrier(simInfo.mnc);
 // Jio: 701, Airtel: 702, Vi: 703, BSNL: 704

 // Generate carrier-specific MMI code
 const getForwardingCode = (carrier, superowlNumber) => {
 switch (carrier) {
 case 'jio': return `*21*${superowlNumber}#`;
 case 'airtel': return `**21*${superowlNumber}#`;
 case 'vi': return `*21*${superowlNumber}#`;
 case 'bsnl': return `*21*${superowlNumber}#`;
 }
 };

 const handleForward = () => {
 const code = getForwardingCode(carrier, businessPhone);
 Linking.openURL(`tel:${code}`); // Opens dialer with MMI code
 };

 return (
 <View>
 <Text>Carrier: {carrier} (SIM {simInfo.slot + 1})</Text>
 <Text>Forward all calls to: {businessPhone}</Text>
 <Button title="Set Up Forwarding" onPress={handleForward} />
 </View>
);
};

```

## Slack Integration — Complete

### slack\_service.py — Block Kit Notifications

```

slack_service.py
async def send_live_call_notification(business, call_id, caller_phone):
 """Send call start notification with action buttons"""
 blocks = [
 {
 "type": "header",
 "text": {"type": "plain_text", "text": "📞 Live Call"}
 },
 {
 "type": "section",
 "fields": [
 {"type": "mrkdwn", "text": f"*Caller:* \n{caller_phone}"},
 {"type": "mrkdwn", "text": f"*Business:* \n{business['display_name']}"},
 {"type": "mrkdwn", "text": f"*Time:* \n{datetime.now().strftime('%H:%M')}"}
]
 },
 {
 "type": "actions",
 "elements": [
 {
 "type": "button",
 "text": {"type": "plain_text", "text": "Take Over"},
 "style": "primary",
 "action_id": "takeover",
 "value": call_id
 },
 {
 "type": "button",
 "text": {"type": "plain_text", "text": "End Call"},
 "style": "danger",
 "action_id": "end_call",
 "value": call_id
 }
]
 }
]

```

```

 },
 {
 "type": "button",
 "text": {"type": "plain_text", "text": "Whisper"},
 "action_id": "whisper",
 "value": call_id
 }
]
}

response = await slack_client.chat.postMessage(
 channel=business["slack_channel_id"],
 blocks=blocks,
 text=f"Live call from {caller_phone}"
)

return response["ts"] # Store this as slack_ts for thread updates

async def stream_transcript_chunk(slack_ts, role, text):
 """Update main message + post to thread"""
 # Post to thread
 await slack_client.chat.postMessage(
 channel=channel,
 thread_ts=slack_ts,
 text=f"*{role}*: {text}"
)

async def mark_call_completed(slack_ts, duration, outcome, summary):
 """Update main message with final summary"""
 await slack_client.chat.update(
 channel=channel,
 ts=slack_ts,
 blocks=[
 {
 "type": "section",
 "text": {
 "type": "mrkdwn",
 "text": f"✅ *Call Ended*\n*Duration:* {duration}s\n*Outcome:* {outcome}\n*Summary:* {summary}"
 }
 }
]
)
)

```

## slack\_events.py — Whisper via Thread

```

slack_events.py — When someone types in a Slack thread, it becomes a whisper
@router.post("/slack/events")
async def slack_events(request: Request):
 payload = await request.json()

 # URL verification challenge
 if payload.get("type") == "url_verification":
 return {"challenge": payload["challenge"]}

 # Thread message = whisper
 event = payload.get("event", {})
 if event.get("type") == "message" and event.get("thread_ts"):
 # Find the call associated with this thread
 call_log = await storage.get_call_by_slack_ts(event["thread_ts"])
 if call_log and call_log["status"] == "in_progress":
 # Send whisper to VAPI
 await vapi_service.send_message(
 call_log["vapi_call_id"],
 {"message": event["text"], "type": "whisper"}
)
)

```

## slack\_commands.py — Slash Commands

```

slack_commands.py
@router.post("/slack/commands")
async def slack_commands(request: Request):
 form = await request.form()
 command = form["command"]
 text = form["text"]
 channel = form["channel_id"]

 # Verify Slack signature
 if not verify_slack_signature(request):
 return {"text": "Invalid signature"}

```



```

if command == "/superowl-whisper":
 # Find active call in this channel
 call = await storage.get_active_call_by_channel(channel)
 if call:
 await vapi_service.send_message(call["vapi_call_id"], {
 "message": text, "type": "whisper"
 })
 return {"text": f"Whispered: {text}"}
 return {"text": "No active call found"}

elif command == "/superowl-end":
 call = await storage.get_active_call_by_channel(channel)
 if call:
 await vapi_service.end_call(call["vapi_call_id"])
 return {"text": "Call ended"}
 return {"text": "No active call found"}

elif command == "/superowl-transfer":
 call = await storage.get_active_call_by_channel(channel)
 if call:
 await vapi_service.transfer_call(call["vapi_call_id"], f"sip:{text}@vapi.ai")
 return {"text": f"Transferring to {text}"}
 return {"text": "No active call found"}

```

## slack\_actions.py — Button Clicks

```

slack_actions.py
@router.post("/slack/actions")
async def slack_actions(request: Request):
 payload = json.loads(await request.form())["payload"]
 action = payload["actions"][0]
 action_id = action["action_id"]
 call_id = action["value"]

 if action_id == "takeover":
 # Transfer call to owner's SIP
 call = await storage.get_call_log_by_id(call_id)
 business = await storage.get_business_by_uuid(call["uuid"])
 owner_sip = f"sip:{business['owner_no']}@vapi.ai"
 await vapi_service.transfer_call(call["vapi_call_id"], owner_sip)
 return {"text": "Call transferred to you"}

 elif action_id == "decline_transfer":
 await vapi_service.send_message(call_id, {
 "message": "I'm sorry, no one is available right now."
 })
 return {"text": "Transfer declined"}

 elif action_id == "end_call":
 await vapi_service.end_call(call_id)
 return {"text": "Call ended"}

 elif action_id == "whisper":
 # Open modal for whisper input
 return {"text": "Type your whisper in the thread below"}

```

## Billing System

```

billing.py
PLANS = {
 "starter": {"credits": 1000, "price": 999},
 "value": {"credits": 2200, "price": 1999},
 "power": {"credits": 4800, "price": 3499},
 "pro": {"credits": 10000, "price": 6999},
}

@router.get("/balance/{uid}")
async def get_balance(uid: str):
 profile = await storage.get_user_profile(uid)
 return {"credits": profile.get("credits", 0)}

@router.post("/purchase/{uid}")
async def purchase(uid: str, plan: str):
 plan_data = PLANS[plan]
 await storage.update_user_profile(uid, {
 "credits": storage.get_user_profile(uid)["credits"] + plan_data["credits"]
 })
 await storage.create_transaction(uid, plan, plan_data["price"])

```

```
 return {"success": True, "credits_added": plan_data["credits"]}

@router.post("/deduct/{uid}")
async def deduct(uid: str, call_id: str):
 # Firebase auth required
 call = await storage.get_call_log_by_id(call_id)
 duration_seconds = call["duration"]
 credits_used = calculate_credits(duration_seconds)

 profile = await storage.get_user_profile(uid)
 new_balance = profile["credits"] - credits_used

 await storage.update_user_profile(uid, {"credits": max(0, new_balance)})
 return {"credits_deducted": credits_used, "remaining": max(0, new_balance)}
```

## Gemini Service — KB → Prompt

```
gemini_service.py
async def generate_business_prompt(knowledge_base_text):
 """
 Takes raw KB text (business description, services, FAQ)
 and generates a structured system prompt for the AI.
 Model: gemini-2.5-flash-lite
 """
 prompt = f"""You are a prompt engineering assistant. Given this business information,
 create a structured system prompt for a voice AI assistant that handles customer calls.

 Business Information:
 {knowledge_base_text}

 Generate a system prompt with these sections:
 1. Role and tone
 2. Business knowledge (services, hours, location)
 3. Common scenarios and how to handle them
 4. Escalation rules (when to transfer to human)
 5. Greeting message

 Output in plain text, ready to use as a system prompt."""

 response = await gemini_client.generate_content(prompt)
 return response.text
```

## Known Issues (From Code Audit)

Issue	Severity	Location	Fix
<code>get_user_profile_by_phone()</code> doesn't exist in <code>json_storage</code>	Critical	<code>json_storage.py:176</code>	Implement the function
<code>voice_id</code> stored but never sent to VAPI	Medium	<code>call_session.py</code>	Add to <code>assistantOverrides</code>
Multiple mobile buttons dead (Cancel Call, End Call)	Medium	mobile screens	Wire up event handlers
Hardcoded keystore password	Security	<code>build.gradle</code>	Move to env var
<code>showIntent</code> variable never set to true	Low	<code>FeedScreen.js</code>	Wire intent display

## Architecture Decisions

- Dual storage (Firestore/JSON):** Production uses Firestore. Dev uses JSON with `fcntl.flock`. The facade ( `storage.py` ) routes transparently. Enables zero-setup local development.
- Fire-and-forget for notifications:** VAPI has 7.5s webhook timeout. Slack/FCM can't be called within that window reliably. Solution: return to VAPI immediately, then send notifications as background tasks.
- WebSocket + FCM dual channel:** WebSocket for high-frequency transcript (needs ordering). FCM for state changes (call started, ended). Different channels for different data patterns.
- 5-layer ANI resolution:** Real-world SIP headers are messy. Different carriers provide different headers. The 5-layer approach handles edge cases: Diversion header, SIP From/To, phone map cache, DB by business phone, DB by owner phone.
- Multi-tenant via nested business object:** Business config lives inside `user_profiles`. No separate businesses collection. Denormalized for fast reads — one document fetch gets everything needed for

call handling.